

Codificarea informației; compresia datelor

Unitatea de învățare nr. 3 · Programarea calculatoarelor și limbaje de programare I

Obiective și cuprins

Obiectivele unității

- Însușirea noțiunilor fundamentale din domeniul **compresiei datelor**
- Familiarizarea cu principalele tehnici de compresie **fără pierderi** și **cu pierderi**
- Timp mediu de studiu: **3–4 ore**; reprezentați individual elementele grafice, nu doar le vizionați

1. Compresia datelor – generalități
2. Aspecte generale privind codificarea
3. Codificarea R-L-E (Run Length Encoding)
4. Codificarea Huffman
5. Codificarea aritmetică
6. Codificarea Shannon-Fano
7. Codificarea informației multimedia; transmiterea datelor în funcție de tip

+ Test de autoevaluare rezolvat pas cu pas · Rezumat

1

Compresia datelor – generalități

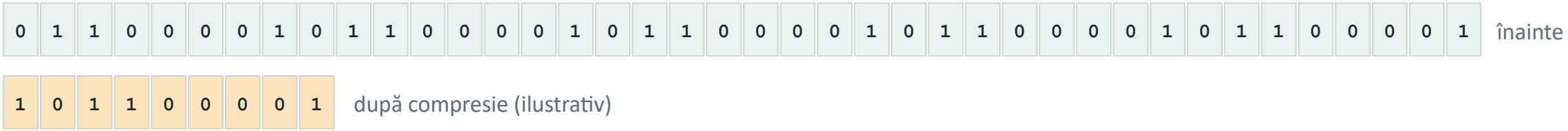
Ce este compresia, pe ce date se aplică, ce principiu folosește, cum se clasifică și cum se măsoară performanța

Ce este compresia datelor?

Compresia datelor = procedura prin care un **flux de date de intrare** (mesajul generat de o sursă) este convertit într-un **flux de ieșire comprimat**, de dimensiune mai mică → mai ușor de memorat sau de transmis.



Reprezentarea fiind binară, scopul este **reducerea numărului de simboluri binare (biți)** necesare reprezentării datelor:



Compresia este de fapt o **codificare**: fiecărei secvențe de date i se asociază (prin mapare) o secvență de cod binar mai scurtă.

Ce tipuri de date de intrare se comprimă?

T Text

- Cod **ASCII** – variante pe **7** și pe **8 biți**
- Cod **EBCDIC** – 8 biți, maxim **256** caractere

I Imagine

- **Bitmap**: matrice de pixeli, fiecare cu codul său de culoare
- **Vectorial**: colecție de linii și curbe conectate

S Sunet

- Sunetul = **undă elastică**
- Reprezentat printr-o **funcție periodică**

M Multimedia

- Combinație de **text + imagine + sunet**
- Volume mari → compresia e esențială

Principiul comun: reducerea redundanței

Toate procedurile de compresie, indiferent de tipul datelor, se bazează pe **reducerea redundanței** din datele de intrare.

Redundanță alfabetică

Unele litere apar **foarte des** într-un text (ex. în română: a, e, i).

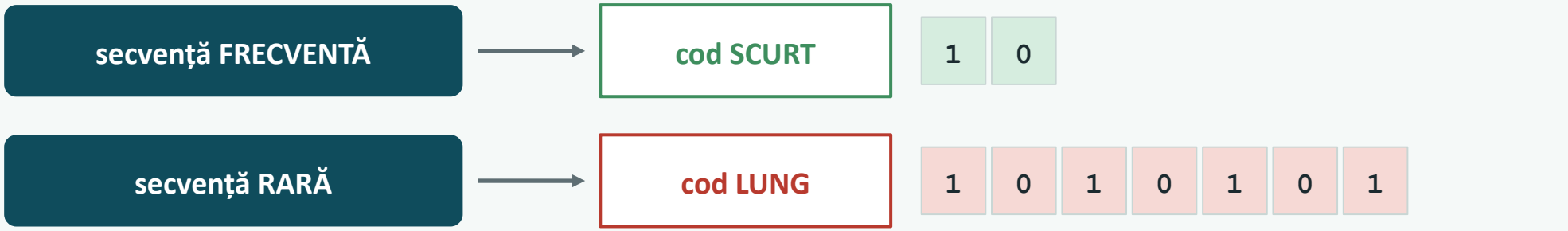
Redundanță contextuală

Grupuri de litere apar frecvent împreună (ex. „ea”, „re”, „ul”).

Redundanță spațială

În imagini, **pixelii adiacenți** sunt corelați (culori identice/apropiate).

Regula de aur a compresiei



*Așa se scurtează **în medie** reprezentarea întregului mesaj.*

Clasificare 1: fără pierderi vs. cu pierderi

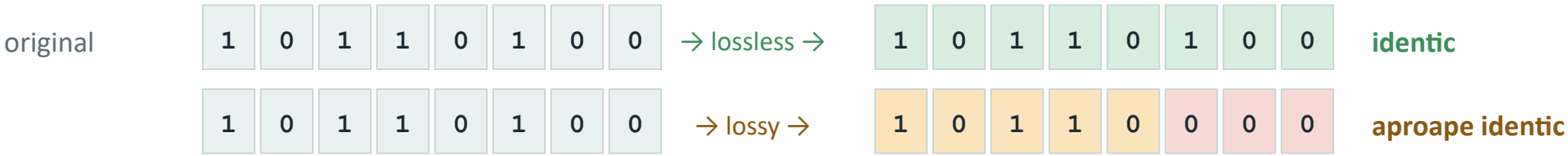
Rezultatul decompresiei poate fi **identic** cu fluxul inițial sau **nu** → două clase de algoritmi:

Fără pierdere de informație (lossless)

- Decompresia reface **exact** datele inițiale
- Folosită când **niciun detaliu** nu poate fi pierdut
- Exemple: **texte**, date numerice, **imagini medicale**, imagini militare, fișiere cu **cod sursă**

Cu pierdere de informație (lossy)

- Se pierd **detalii nesemnificative**
- Oferă în general un **grad de compresie mai mare**
- Exemple: **sunete și imagini**, unde o pierdere minimă de rezoluție este insesizabilă sau acceptabilă



Clasificare 2: compresie simetrică vs. asimetrică

Simetrică

- Compresia și decompresia folosesc practic **același algoritm**, aplicat **în sens opus**
- Aceeași complexitate → **același timp** de compresie și decompresie

Asimetrică

- Compresia și decompresia implică **resurse și complexități diferite**
- De obicei **compresia este mai solicitantă**
- Multe metode moderne sunt asimetrice – ex. **compresia video**





Lungimea barelor = efortul (timp/complexitate) – reprezentare schematică

Măsurarea performanței: raport și factor de compresie

Raport de compresie

raport = dimensiunea datelor **comprimate** / dimensiunea datelor **necomprimate**

Factor de compresie

factor = 1 / raport → cu cât e **mai mare**, cu atât compresia e **mai bună**

Exemplu din curs: raport = 0.7



Datele ocupă după compresie **70%** din dimensiunea inițială; factorul = 1/0.7 ≈ **1.43**.

Beneficiile compresiei

1. Reducerea spațiului de depozitare a datelor
2. Utilizarea eficientă a lățimii de bandă; rată de transfer mai mare
3. Timp de transfer redus (volum de informație mai mic)

2

Aspecte generale privind codificarea

Alfabet, cuvânt de cod, lungime fixă vs. variabilă, lungime medie, proprietățile codurilor, informație și entropie

Alfabetul sursei și cuvântul de cod

O sursă discretă generează simboluri dintr-un **alfabet** $S = \{s_1, s_2, \ldots, s_n\}$ – mulțimea tuturor simbolurilor posibile.

n = 26

s_i = literele alfabetului

n = 10

s_i = cifrele sistemului zecimal

Codarea: fiecare simbol s_k este reprezentat printr-un **șir de biți = cuvânt de cod**, folosind cât mai puțini biți.

Alfabet cu **4** simboluri → **2 biți**/simbol

s_1	s_2	s_3	s_4
0 0	0 1	1 0	1 1

Alfabet cu **16** simboluri → **4 biți**/simbol: 0000, 0001, …, 1111

Regula generală (lungime fixă): pentru n simboluri sunt necesari **$\log_2 n$** biți (rotunjit în sus când n nu e putere a lui 2: $\lceil \log_2 n \rceil$). Ex.: 26 litere → $\lceil 4.70 \rceil$ = **5 biți**; 10 cifre → $\lceil 3.32 \rceil$ = **4 biți**.

Observație: codificarea cu lungime fixă **nu este indicată** când simbolurile au probabilități de apariție diferite → se preferă **lungime variabilă**.

Lungime fixă vs. lungime variabilă – exemplul din curs

Alfabet cu 8 simboluri $\{s_1, \ldots, s_8\}$, cu probabilități diferite. Simbolurile frecvente primesc coduri scurte:

Simbol	$p(s_k)$	Cod lungime fixă	Cod lungime variabilă	$l(s_k)$
s_1	5/18	000	**0**	1
s_2	4/18	001	**01**	2
s_3	3/18	010	**10**	2
s_4	2/18	100	**11**	2
s_5	1/18	011	**010**	3
s_6	1/18	110	**001**	3
s_7	1/18	101	**100**	3
s_8	1/18	111	**110**	3

Ideea

Cu cât **p** e mai mare, cu atât **l** e mai mic: s_1 (5/18) → 1 bit; $s_5 \ldots s_8$ (1/18) → 3 biți.

Lungime fixă

8 simboluri → $\log_2 8$ = **3 biți** pentru fiecare simbol, indiferent de frecvență.
Cât câștigăm cu lungimea variabilă? → slide-ul următor.

Calculul lungimii medii a cuvântului de cod – pas cu pas

Formula (1): $\bar{l} = \Sigma_{k=1..n}\, p(s_k) \cdot l(s_k)$ (numărul mediu de biți per simbol, ponderat cu probabilitățile)

1 **Înmulțim $p(s_k) \cdot l(s_k)$ pentru fiecare simbol:**

s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8
5/18·1 **= 5/18**	4/18·2 **= 8/18**	3/18·2 **= 6/18**	2/18·2 **= 4/18**	1/18·3 **= 3/18**	1/18·3 **= 3/18**	1/18·3 **= 3/18**	1/18·3 **= 3/18**

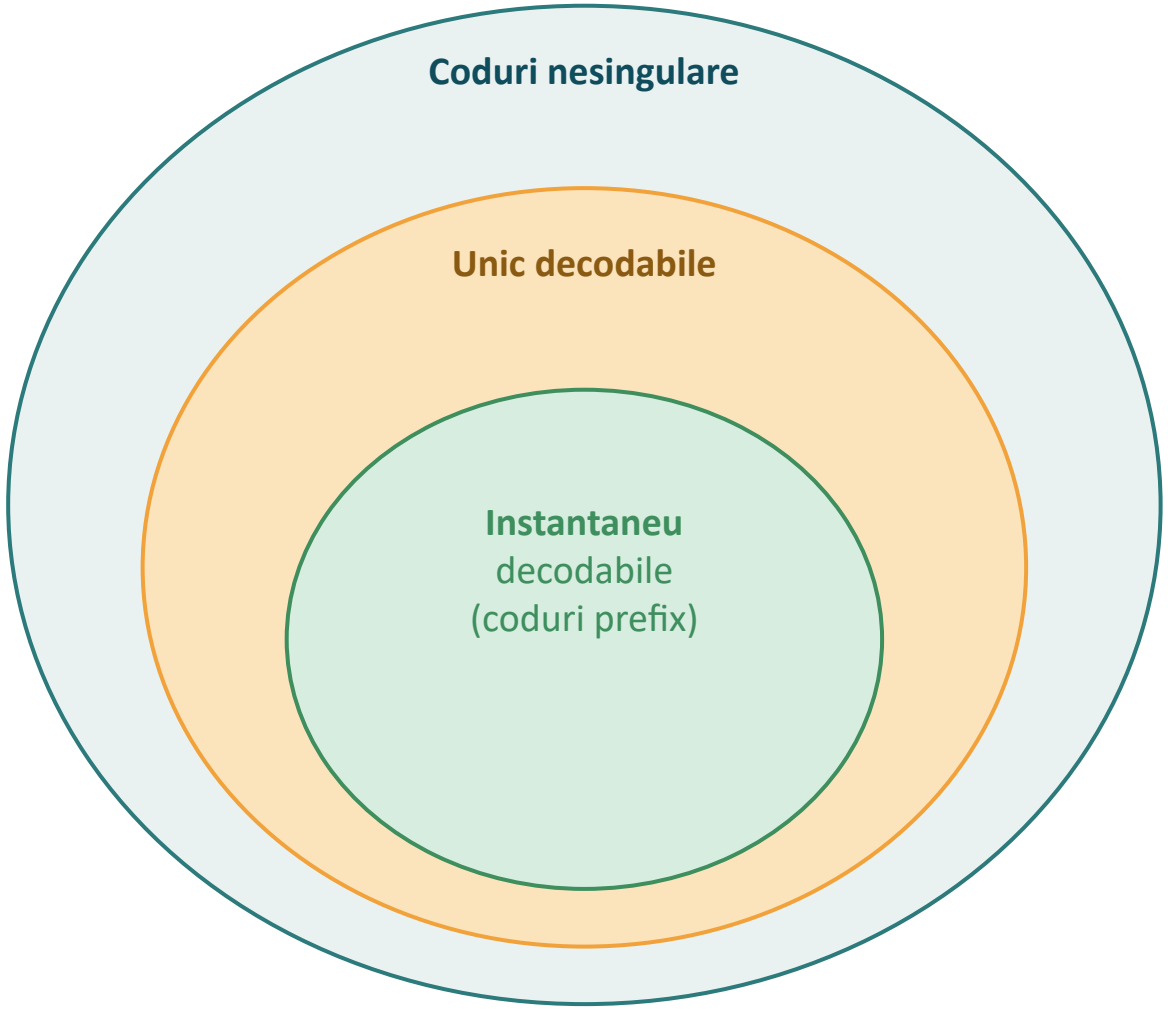
2 **Adunăm:** (5 + 8 + 6 + 4 + 3 + 3 + 3 + 3) / 18 = **35/18 ≈ 1.94 biți/simbol**

3 **Comparăm:**



Observație: pentru alfabete cu foarte multe simboluri, diferența față de codul cu lungime fixă devine **semnificativă**.

Proprietățile unui cod



1 Nesingular

Un cod este nesingular dacă **toate** cuvintele de cod sunt **distincte**.

2 Unic decodabil

Un cod este unic decodabil dacă două șiruri de simboluri **distincte** nu generează **același șir de biți**.

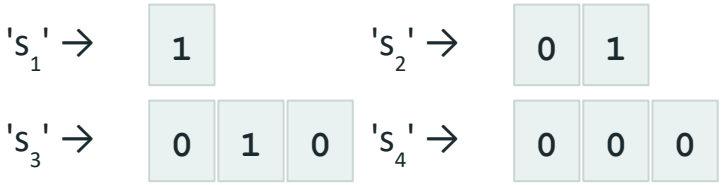
3 Instantaneu decodabil

Un cod este instantaneu decodabil dacă **niciun** cuvânt de cod nu este **prefix** pentru alt cuvânt de cod. Dacă $c = x_1x_2\ldots x_n$ e cuvânt de cod, atunci $x_1\ldots x_k$ $(1 \le k < n)$ **nu** e cuvânt de cod (**proprietatea de prefix**).

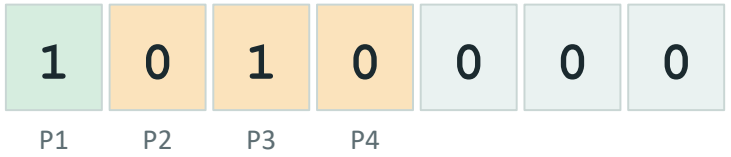
Fiecare clasă e inclusă în cea dinaintea ei.

Exemplu: decodarea șirului 1010000 – pas cu pas

Cod pentru 4 simboluri:



Șirul recepționat:



1	Recepționează **1**	Singurul cod ce începe cu 1 este s_1 → identificat **sigur**: s_1
2	Recepționează **0**	Poate fi începutul lui s_2 (01), s_3 (010) sau s_4 (000)
3	Recepționează **1** (avem 01)	Opțiunile se restrâng la s_2 sau s_3
4	Recepționează **0** (avem 010)	**Blocaj**: 010 poate fi s_3 **sau** s_2 (01) urmat de începutul unui nou simbol

Atenție: **01** (s_2) este **prefix** al lui **010** (s_3) → codul **nu este instantaneu decodabil**: decodorul nu poate decide pe loc. Mai mult, **0101** se poate citi ca s_2s_2 (01|01) sau s_3s_1 (010|1) → codul **nu este nici unic decodabil**.

Ce înseamnă un cod „bun”?

Unic decodabil
orice șir de biți are o singură interpretare

+

Instantaneu decodabil
fiecare simbol se recunoaște imediat ce i s-a primit ultimul bit

+

Lungime medie minimă
$\bar{l}$ calculat cu formula (1) să fie cât mai mic

Pentru a compara două scheme de codificare trebuie măsurată **cantitatea de informație** dintr-un mesaj → teoria informației (Shannon).

Informația unui simbol

$i(s_k) = -\log_2 p(s_k) = \log_2(1/p(s_k))$ [biți]

Cu cât **probabilitatea e mai mare**, cu atât **informația e mai mică**.

Entropia informațională a sursei

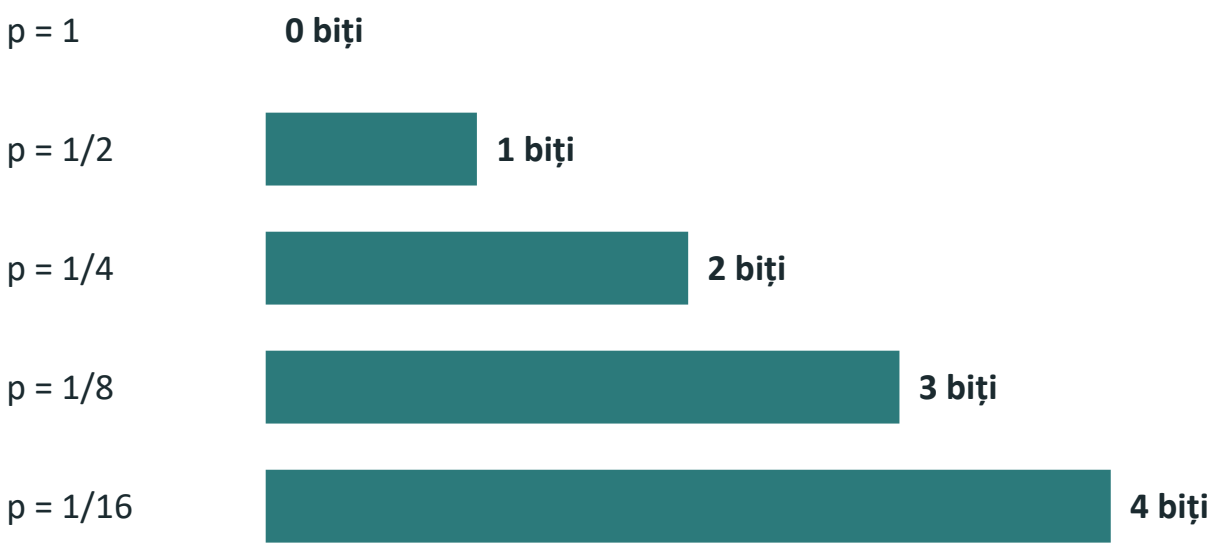
Informația **medie** generată de sursă:

$H(S) = \Sigma\, p(s_k) \cdot \log_2(1/p(s_k))$ (2)

Informația unui simbol: $i(s) = \log_2(1/p)$

p(s)	1/p(s)	$i(s) = \log_2(1/p)$ [biți]
1	1/1	0
1/2	2	1
1/4	4	2
1/8	8	3
1/16	16	4

Informația crește când probabilitatea scade:



Observație: un eveniment **sigur** (p = 1) nu aduce **nicio** informație (0 biți); un eveniment **rar** aduce **multă** informație. De aceea simbolurile rare „merită” coduri mai lungi.

Calculul entropiei – exemplul din curs, pas cu pas

Sursa: s_1=0.0625, s_2=0.0625, s_3=0.125, s_4=0.25, s_5=0.5

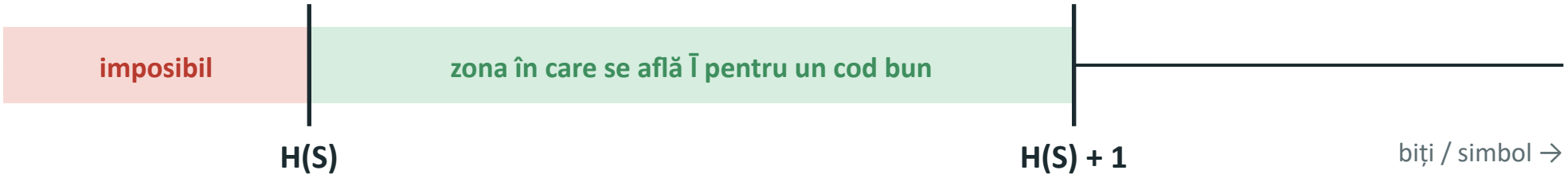
Simbol	$p(s_k)$	Pas 1: 1/p	Pas 2: $\log_2(1/p)$	Pas 3: $p \cdot \log_2(1/p)$
s_1	0.0625	$16 = 2^4$	4	$0.0625 \cdot 4 = 0.25$
s_2	0.0625	$16 = 2^4$	4	$0.0625 \cdot 4 = 0.25$
s_3	0.125	$8 = 2^3$	3	$0.125 \cdot 3 = 0.375$
s_4	0.25	$4 = 2^2$	2	$0.25 \cdot 2 = 0.5$
s_5	0.5	$2 = 2^1$	1	$0.5 \cdot 1 = 0.5$
Pas 4: Σ	1			**H(S) = 1.875 biți**

Observație: probabilitățile sunt puteri ale lui 1/2, deci logaritmii sunt **numere întregi** ($1/0.0625 = 16 = 2^4 \rightarrow \log_2 16 = 4$). Interpretare: **niciun** cod nu poate folosi, în medie, mai puțin de **1.875 biți/simbol** pentru această sursă.

Relația dintre entropie și lungimea medie

Teoremă: $H(S) \le \bar{l} < H(S) + 1$

Entropia este **limita inferioară** a lungimii medii; un cod bine ales rămâne la **mai puțin de 1 bit** peste ea.



Verificare pe exemplu (H = 1.875)

Alegem $l(s_k) = \lceil \log_2(1/p) \rceil = 4, 4, 3, 2, 1 \rightarrow \bar{l} = 0.25+0.25+0.375+0.5+0.5 = \mathbf{1.875} = H(S)$ (caz ideal).

Pe exemplul Huffman (mai departe)

$H = 3.25$, $\bar{l} = 3.30 \rightarrow 3.25 \le 3.30 < 4.25$ ✓

Demonstrație (1): de ce $H(S) \le \bar{l}$

1. $H(S) - \bar{l} = \Sigma\, p(s_k)\, \log_2(1/p(s_k)) - \Sigma\, p(s_k)\, l(s_k)$

2. $= \Sigma\, p(s_k)\, [\, \log_2(1/p(s_k)) - l(s_k)\,]$ (scoatem $p(s_k)$ factor comun)

3. $= \Sigma\, p(s_k)\, [\, \log_2(1/p(s_k)) - \log_2 2^{l(sk)}\,]$ (scriem $l = \log_2 2^l$)

4. $= \Sigma\, p(s_k) \cdot \log_2(\, 2^{-l(sk)} / p(s_k)\,)$ (diferență de logaritmi)

5. $\le \log_2(\, \Sigma\, p(s_k) \cdot 2^{-l(sk)} / p(s_k)\,) = \log_2(\, \Sigma\, 2^{-l(sk)}\,)$ (inegalitatea (3))

6. $\le \log_2 1 = 0$ ($\Sigma\, 2^{-l} \le 1$, Kraft)

Inegalitatea (3) – funcții concave

$\Sigma\, p_k\, f(x_k) \le f(\, \Sigma\, p_k\, x_k\,)$; $\log_2$ este concavă.

Concluzie: deci $H(S) - \bar{l} \le 0$, adică $\bar{l} \ge H(S)$.

Demonstrație (2): de ce $\bar{l} < H(S) + 1$

1 **Alegem lungimile**

$l(s_k) = \lceil \log_2(1/p(s_k)) \rceil$ (cel mai mic întreg ≥ $\log_2(1/p)$)

2 **Proprietatea rotunjirii**

$\lceil x \rceil < x + 1 \Rightarrow l(s_k) < \log_2(1/p(s_k)) + 1$

3 **Înmulțim cu $p(s_k)$**

$p(s_k)\, l(s_k) < p(s_k) \log_2(1/p(s_k)) + p(s_k)$

4 **Adunăm după k**

$\bar{l} < H(S) + \Sigma\, p(s_k) = H(S) + 1$ (probabilitățile însumează 1)

Concluzie: aceste lungimi respectă $\Sigma\, 2^{-l} \leq \Sigma\, p = 1$, deci există un cod prefix cu ele. Concluzie: **$H(S) \leq \bar{l} < H(S) + 1$** (formula (4) din curs).

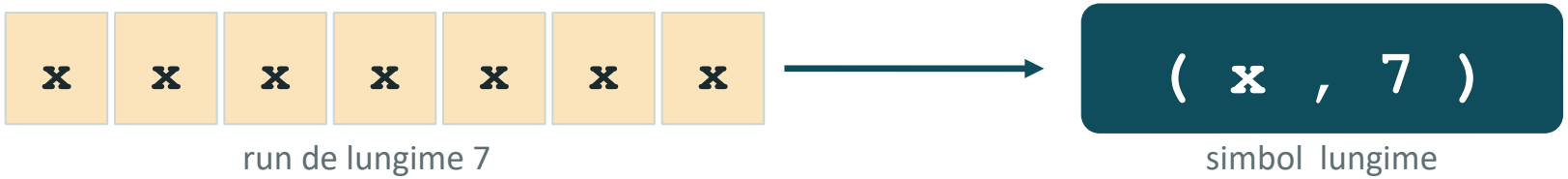
3

Codificarea R-L-E (Run Length Encoding)

Unul dintre cei mai simpli algoritmi de compresie – pentru date ce conțin simboluri repetitive

Principiul RLE

O **secvență repetitivă** („run”) de caractere, biți sau pixeli cu aceeași valoare se înlocuiește cu **o singură apariție + lungimea secvenței**. Caracterele care nu fac parte din nicio repetare rămân neprelucrate.



R Run

repetarea unui simbol

L Run-Length

numărul de simboluri **consecutive** care au aceeași valoare

✓ Când funcționează bine

Când datele au **secvențe lungi** de valori identice (imagini cu zone uniforme, fișiere cu multe zerouri, spații).

Exemplul 1 (text): aaaaacccbbaaabbbccccccc – pas cu pas

Pas 1. Parcurgem șirul și grupăm caracterele identice consecutive:



Pas 2. Pentru fiecare grup scriem perechea (caracter, lungime):



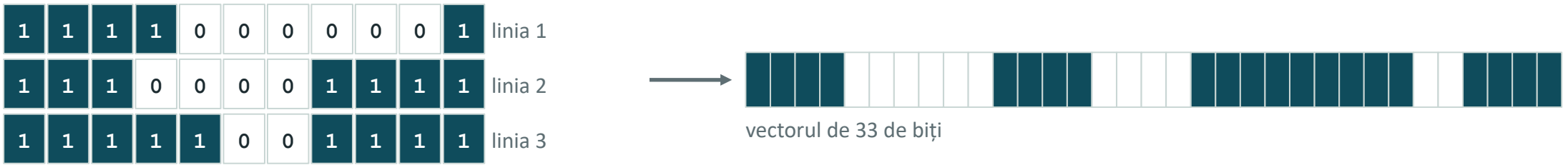
Pas 3. Rezultatul: (a,5), (c,3), (b,2), (a,3), (b,3), (c,6) → 22 caractere devin 6 perechi.

Atenție: în curs apare **(a,4)** pentru primul grup, dar șirul începe cu **5** de „a” (aaaaa) → perechea corectă este **(a,5)**. Verificare: 5+3+2+3+3+6 = 22 = lungimea șirului.

Decodare: fiecare pereche (c, n) se rescrie ca n copii ale lui c.

Exemplul 2 (imagine binară) – pas cu pas

Pas 1. Matricea binară a imaginii se transformă într-un **vector** prin **concatenarea liniilor**:



(împărțire în linii doar ilustrativă)

Pas 2. Identificăm secvențele repetitive (runs) și lungimile lor:



Pas 3. Șirul codificat = **primul element** al vectorului + **lungimile** tuturor secvențelor:



Decodare: pornim cu bitul 1 → 4 de „1”, apoi alternăm: 6 de „0”, 4 de „1”, 4 de „0”, 9 de „1”, 2 de „0”, 4 de „1”. Rezultat: **14644924**.

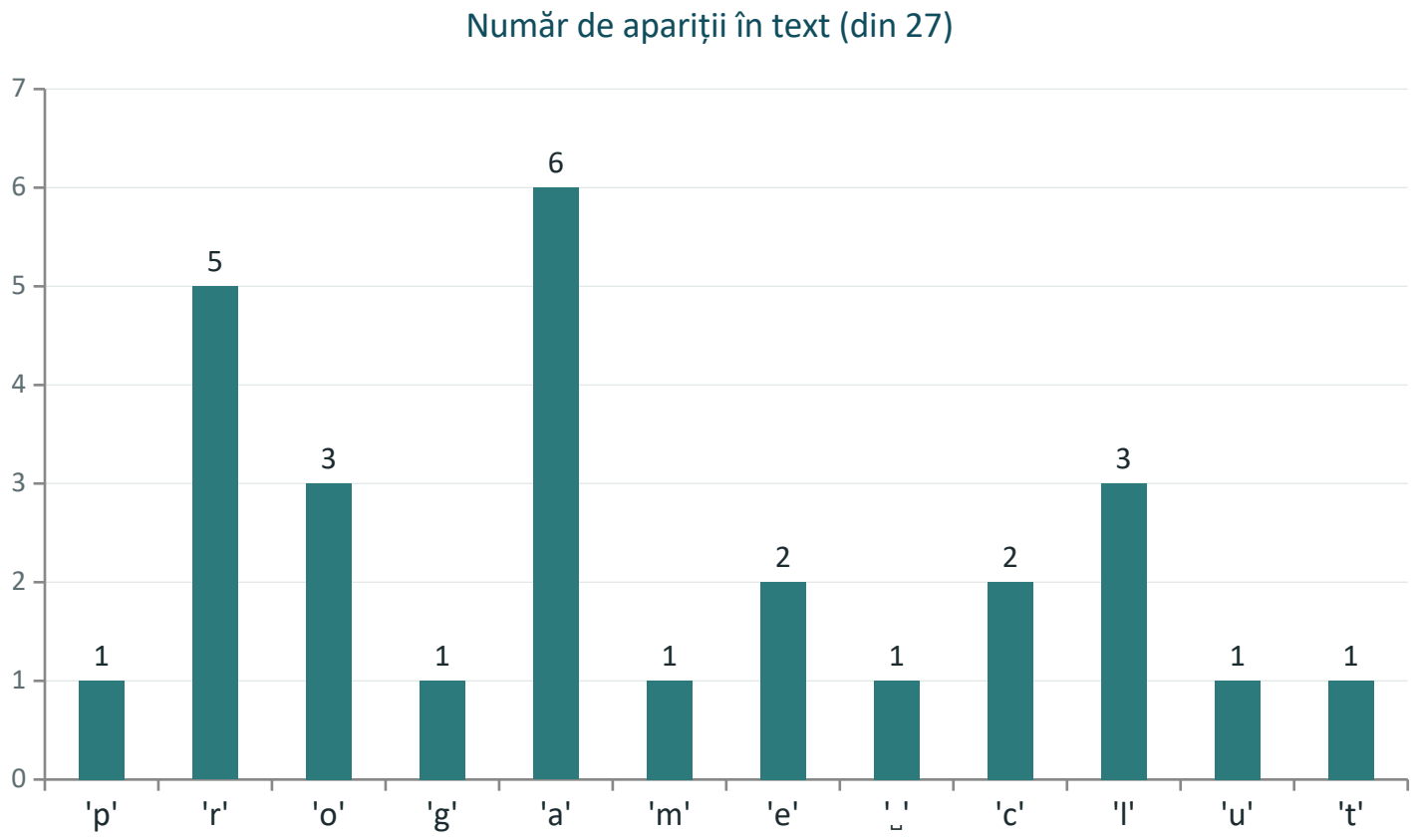
4

Codificarea Huffman

Construcția „bottom-up” a unui arbore binar pe baza frecvențelor de apariție – exemplul „programarea calculatoarelor”

Pasul 0: frecvențele simbolurilor din text

Textul **„programarea calculatoarelor”** are **27** de caractere (inclusiv spațiul). Alfabetul are **12** simboluri; probabilitatea = frecvența / 27.



Simbol	p
'a'	6/27
'r'	5/27
'o', 'l'	3/27 fiecare
'e', 'c'	2/27 fiecare
'p','g','m','␣','u','t'	1/27 fiecare

␣ = caracterul spațiu. Total: 6+5+3+3+2+2+6·1 = 27 ✓

Algoritmul Huffman – schema

1 **Lista inițială**

Fiecare simbol devine o **frunză** cu probabilitatea sa; lista se ordonează **crescător** după probabilitate.

2 **Selectare**

Se aleg **primele două** noduri din listă (cele cu probabilitățile **cele mai mici**).

3 **Unire**

Se creează un **nod nou** cu probabilitatea = **suma** celor două; ele devin descendentul stâng și drept.

4 **Reinserare**

Cele două se **șterg** din listă; nodul nou se inserează pe poziția corectă (lista rămâne ordonată).

5 **Repetare**

Pașii 2–4 se repetă până rămâne **un singur nod** = **rădăcina** (pentru n simboluri: n − 1 uniri).

6 **Alocare coduri**

Ramura stângă primește **0**, cea dreaptă **1**. Codul unui simbol = biții de pe drumul **rădăcină → frunză**.

Arborele se construiește de jos în sus („bottom-up”); pașii 2–5 sunt bucla algoritmului.

Construcția arborelui Huffman – unirea 1 din 11

Lista ordonată crescător:



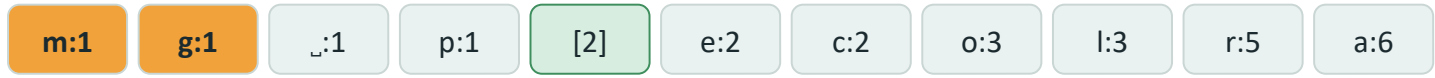
valori în /27; [x] = nod intern; portocaliu = se unesc acum; verde = nod creat la pasul anterior



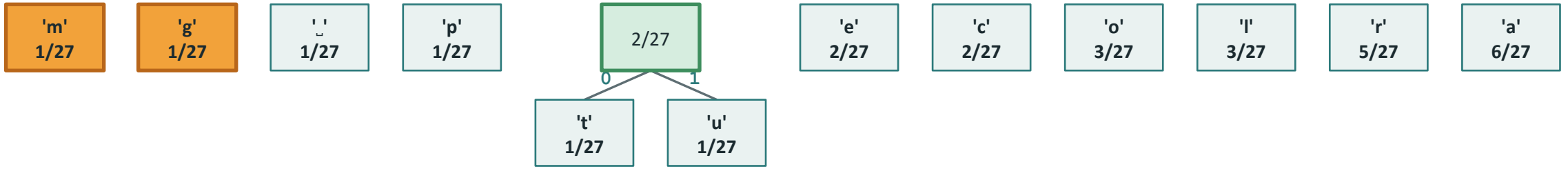
1 Cele mai mici probabilități: 't' (1/27) și 'u' (1/27). Se unesc → nod nou **2/27** (1+1). Stânga (0): 't', dreapta (1): 'u'.

Construcția arborelui Huffman – unirea 2 din 11

Lista ordonată crescător:



valori în /27; [x] = nod intern; portocaliu = se unesc acum; verde = nod creat la pasul anterior



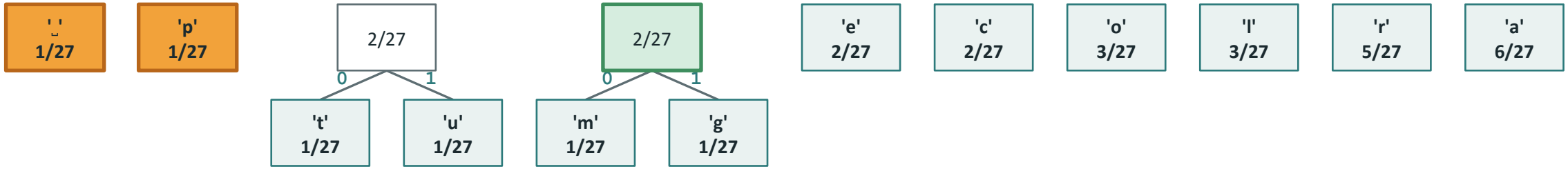
2 Cele mai mici probabilități: 'm' (1/27) și 'g' (1/27). Se unesc → nod nou **2/27** (1+1). Stânga (0): 'm', dreapta (1): 'g'.

Construcția arborelui Huffman – unirea 3 din 11

Lista ordonată crescător:



valori în /27; [x] = nod intern; portocaliu = se unesc acum; verde = nod creat la pasul anterior



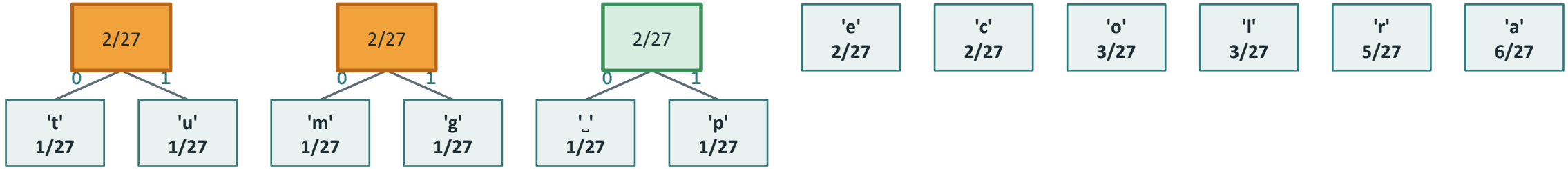
3 Cele mai mici probabilități: '␣' (1/27) și 'p' (1/27). Se unesc → nod nou **2/27** (1+1). Stânga (0): '␣', dreapta (1): 'p'.

Construcția arborelui Huffman – unirea 4 din 11

Lista ordonată crescător:



valori în /27; [x] = nod intern; portocaliu = se unesc acum; verde = nod creat la pasul anterior

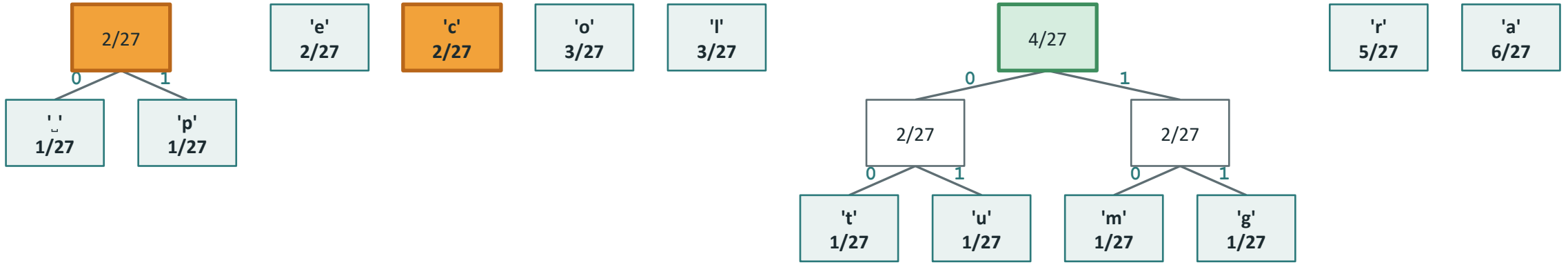


4 Cele mai mici probabilități: nodul 2/27 și nodul 2/27. Se unesc → nod nou **4/27** (2+2). Stânga (0): nodul 2/27, dreapta (1): nodul 2/27.

Construcția arborelui Huffman – unirea 5 din 11

Lista ordonată crescător: **[2]** | e:2 | **c:2** | o:3 | l:3 | [4] | r:5 | a:6

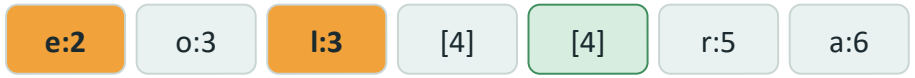
valori în /27; [x] = nod intern; portocaliu = se unesc acum; verde = nod creat la pasul anterior



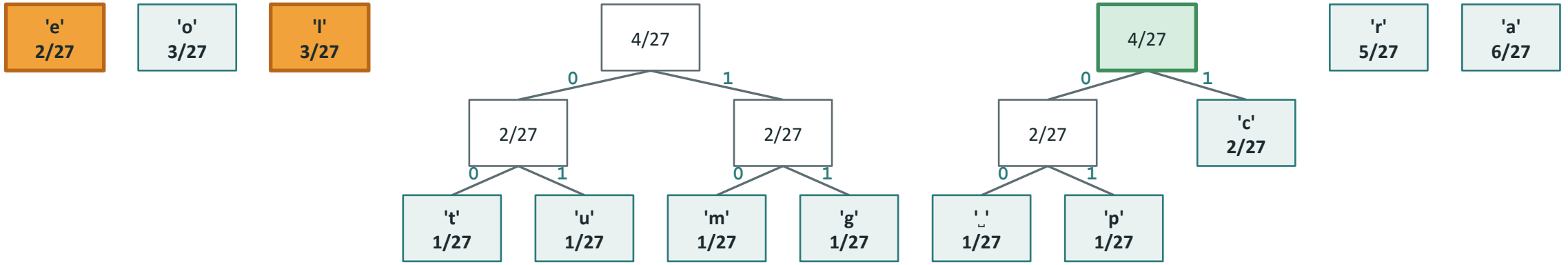
5 Cele mai mici probabilități: nodul 2/27 și 'c' (2/27). Se unesc → nod nou **4/27** (2+2). Stânga (0): nodul 2/27, dreapta (1): 'c'.

Construcția arborelui Huffman – unirea 6 din 11

Lista ordonată crescător:



valori în /27; [x] = nod intern; portocaliu = se unesc acum; verde = nod creat la pasul anterior

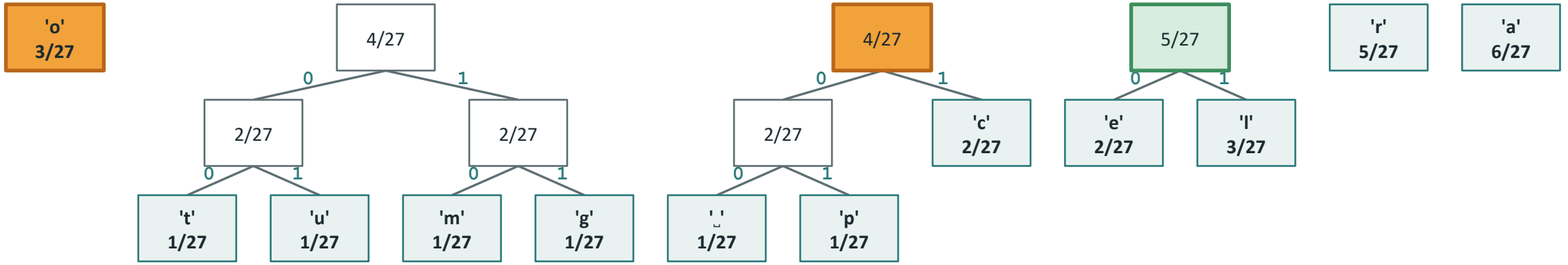


6 Cele mai mici probabilități: 'e' (2/27) și 'l' (3/27). Se unesc → nod nou **5/27** (2+3). Stânga (0): 'e', dreapta (1): 'l'.

Construcția arborelui Huffman – unirea 7 din 11

Lista ordonată crescător: **o:3** | [4] | **[4]** | [5] | r:5 | a:6

valori în /27; [x] = nod intern; portocaliu = se unesc acum; verde = nod creat la pasul anterior

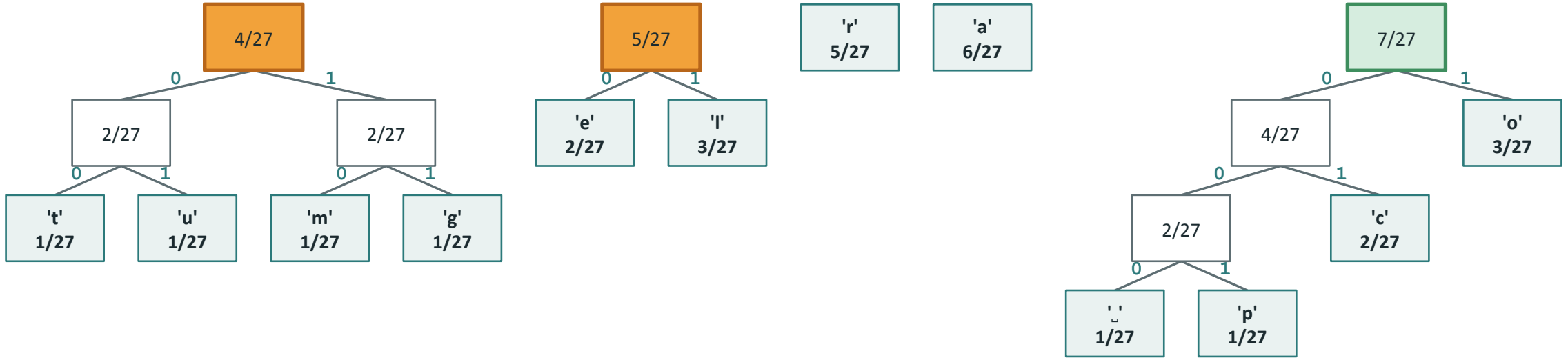


7 Cele mai mici probabilități: nodul 4/27 și 'o' (3/27). Se unesc → nod nou **7/27** (4+3). Stânga (0): nodul 4/27, dreapta (1): 'o'.

Construcția arborelui Huffman – unirea 8 din 11

Lista ordonată crescător: **[4]** **[5]** r:5 a:6 [7]

valori în /27; [x] = nod intern; portocaliu = se unesc acum; verde = nod creat la pasul anterior



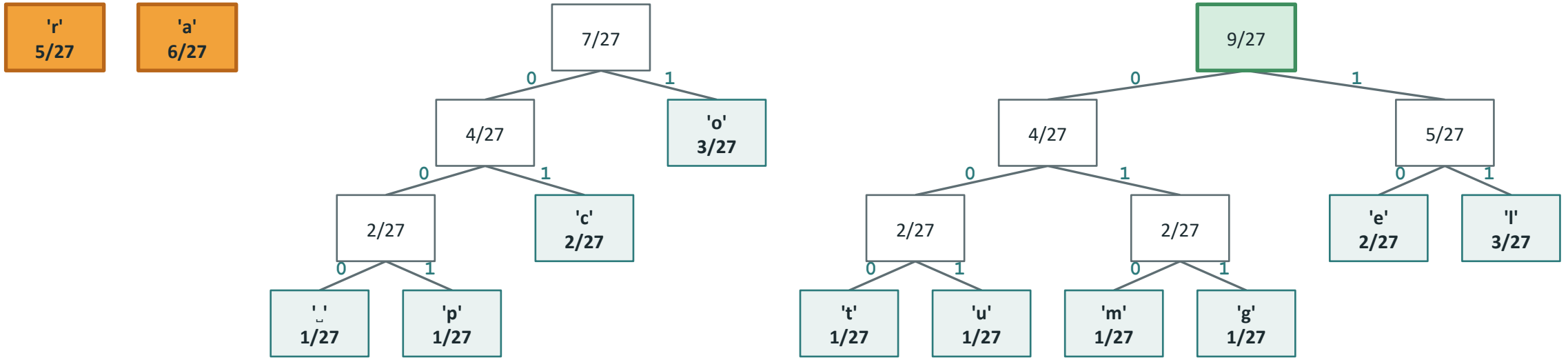
8 Cele mai mici probabilități: nodul 4/27 și nodul 5/27. Se unesc → nod nou **9/27** (4+5). Stânga (0): nodul 4/27, dreapta (1): nodul 5/27.

Construcția arborelui Huffman – unirea 9 din 11

Lista ordonată crescător:



valori în /27; [x] = nod intern; portocaliu = se unesc acum; verde = nod creat la pasul anterior

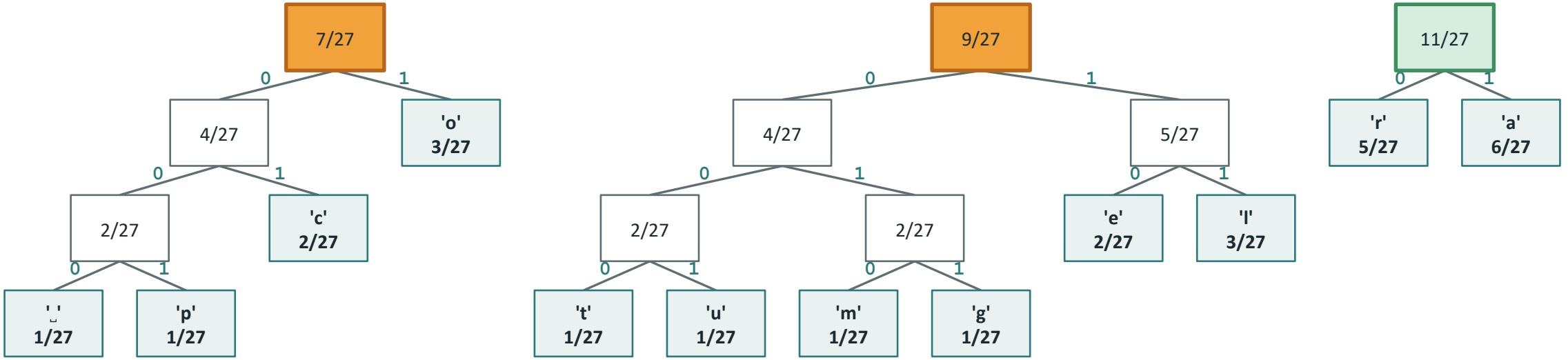


9 Cele mai mici probabilități: 'r' (5/27) și 'a' (6/27). Se unesc → nod nou **11/27** (5+6). Stânga (0): 'r', dreapta (1): 'a'.

Construcția arborelui Huffman – unirea 10 din 11

Lista ordonată crescător: **[7]** **[9]** [11]

valori în /27; [x] = nod intern; portocaliu = se unesc acum; verde = nod creat la pasul anterior

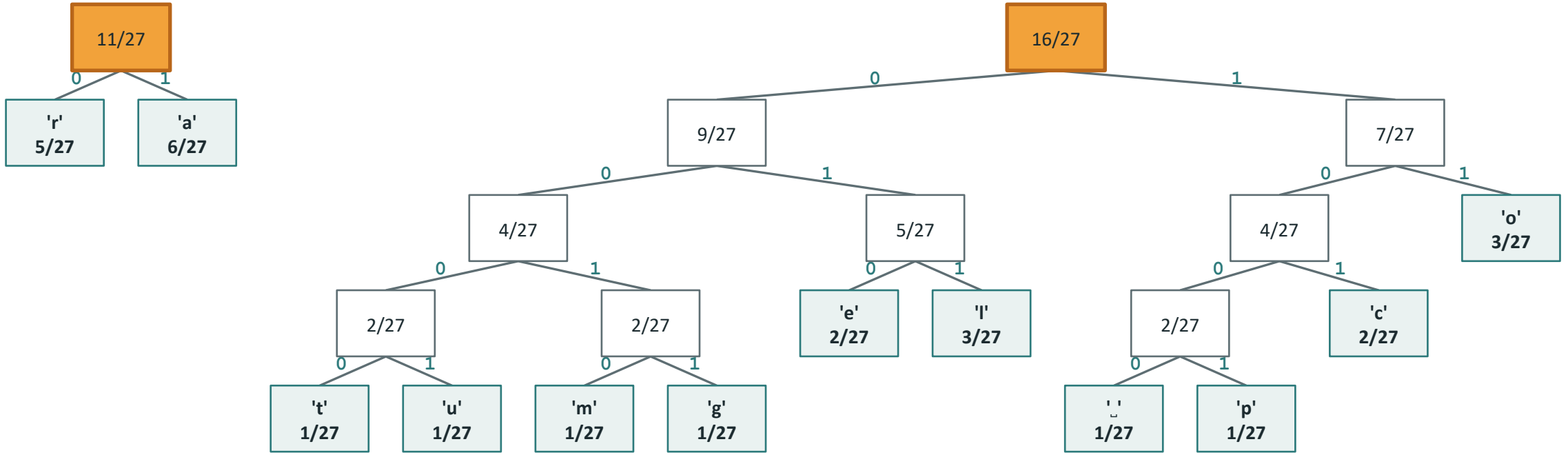


10 Cele mai mici probabilități: nodul 9/27 și nodul 7/27. Se unesc → nod nou **16/27** (9+7). Stânga (0): nodul 9/27, dreapta (1): nodul 7/27.

Construcția arborelui Huffman – unirea 11 din 11

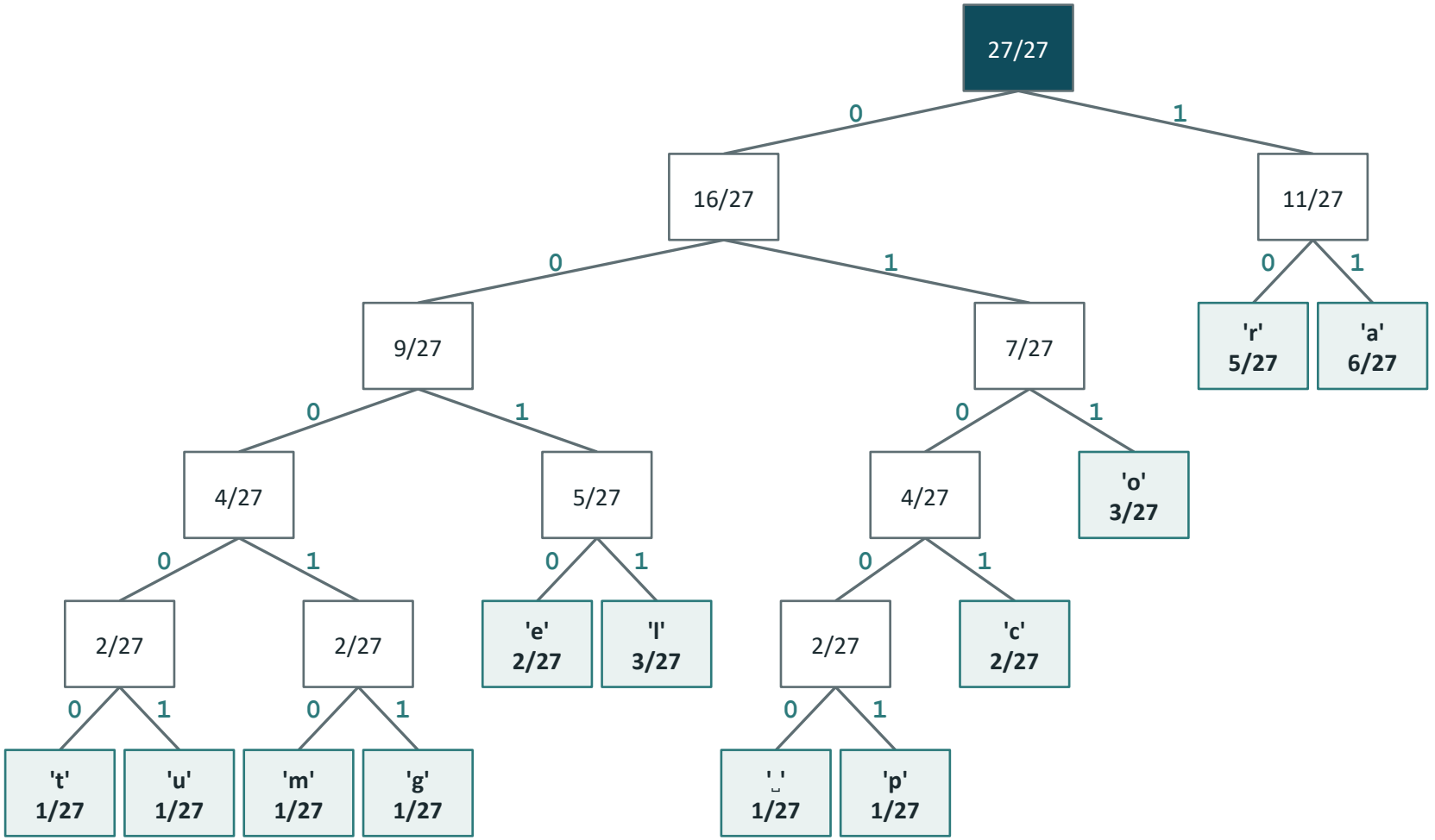
Lista ordonată crescător: **[11]** **[16]**

valori în /27; [x] = nod intern; portocaliu = se unesc acum; verde = nod creat la pasul anterior



11 Au rămas două noduri: 16/27 și 11/27. Unirea lor dă **rădăcina 27/27 = 1**. Arborele este complet (11 uniri = 12 − 1).

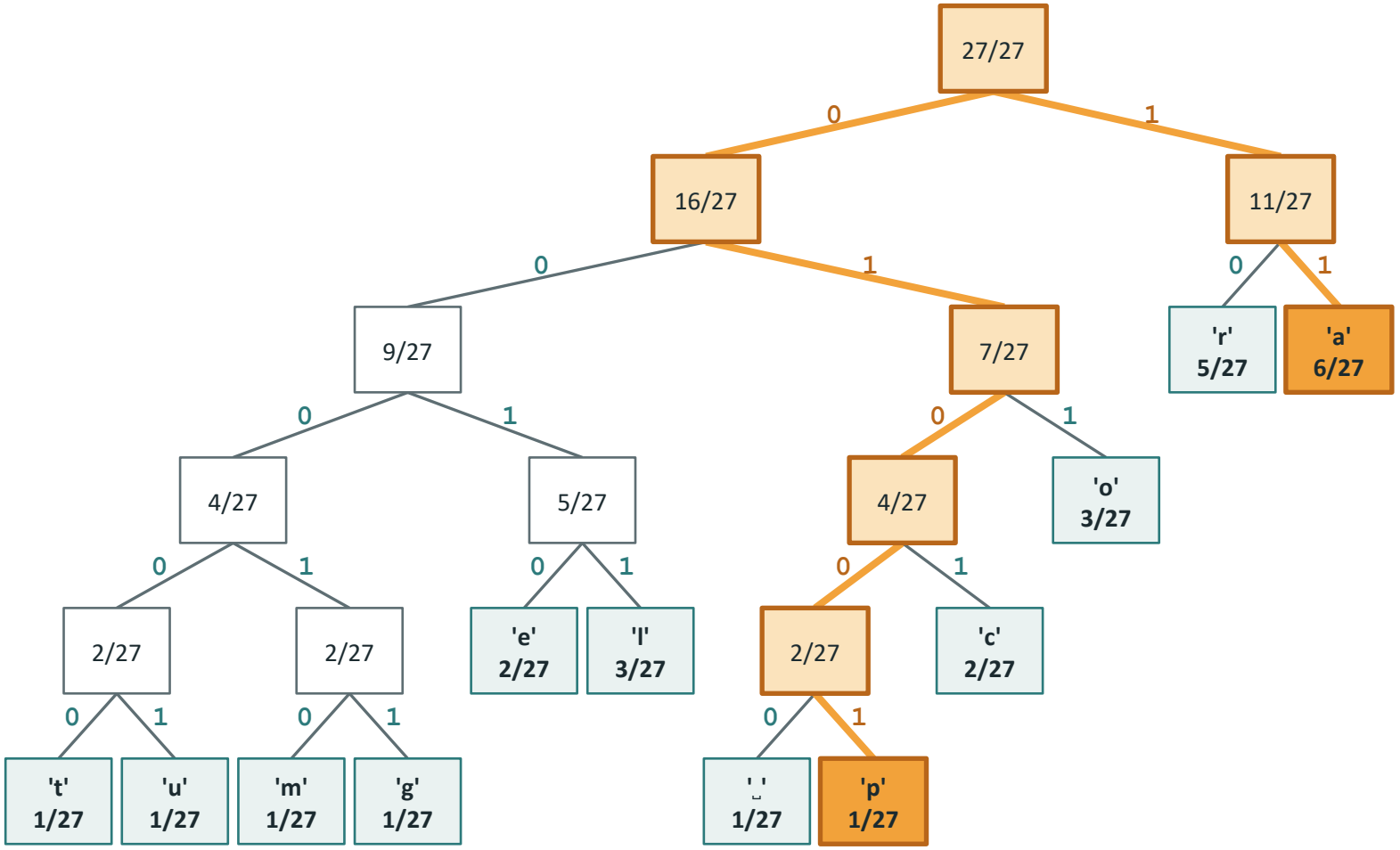
Arborele Huffman final și cuvintele de cod



Simbol	p	Cod
'a'	6/27	11
'r'	5/27	10
'o'	3/27	011
'l'	3/27	0011
'e'	2/27	0010
'c'	2/27	0101
'p'	1/27	01001
'␣'	1/27	01000
'g'	1/27	00011
'm'	1/27	00010
'u'	1/27	00001
't'	1/27	00000

Codul = biții citiți pe drumul de la rădăcină la frunză (stânga = 0, dreapta = 1).

Citirea unui cod din arbore – exemplu pentru 'p' și 'a'



Drumul spre 'p'

rădăcină → 16/27	0
16/27 → 7/27	1
7/27 → 4/27	0
4/27 → 2/27	0
2/27 → 'p'	1

Cod('p') = **01001** (5 biți)

Drumul spre 'a'

rădăcină → 11/27	1
11/27 → 'a'	1

Cod('a') = **11** (2 biți)

'a' e frecvent (6/27) → aproape de rădăcină → cod scurt; 'p' e rar (1/27) → adânc → cod lung.

Lungimea medie a codului Huffman – pas cu pas

Simbol	'p'	'r'	'o'	'g'	'a'	'm'	'e'	'␣'	'c'	'l'	'u'	't'
p (/27)	1	5	3	1	6	1	2	1	2	3	1	1
l (biți)	5	2	3	5	2	5	4	5	4	4	5	5
p·l (/27)	**5**	**10**	**9**	**5**	**12**	**5**	**8**	**5**	**8**	**12**	**5**	**5**

1 **Adunăm termenii:** 5 + 10 + 9 + 5 + 12 + 5 + 8 + 5 + 8 + 12 + 5 + 5 = **89**

2 **Împărțim la 27:** $\bar{l}$ = 89 / 27 ≈ **3.29 biți/simbol**

3 **Comparăm:** 12 simboluri cu lungime fixă → $\lceil \log_2 12 \rceil$ = **4 biți/simbol**

Text întreg: Huffman = 89 biți; lungime fixă = 27 · 4 = 108 biți → economie ≈ 18%.

(curs: 89/27 = 3.29; valoarea exactă 3.296…)

Entropia sursei „programarea calculatoarelor” – pas cu pas

Simbol(uri)	p	1/p	$\log_2(1/p)$	nr. simb.	contribuție = nr · p · $\log_2(1/p)$
'a'	6/27	27/6 = 4.5	2.170	1	1 · 6/27 · 2.170 = 0.482
'r'	5/27	27/5 = 5.4	2.433	1	1 · 5/27 · 2.433 = 0.451
'o', 'l'	3/27	27/3 = 9	3.170	2	2 · 3/27 · 3.170 = 0.704
'e', 'c'	2/27	27/2 = 13.5	3.755	2	2 · 2/27 · 3.755 = 0.556
6 simboluri rare	1/27	27	4.755	6	6 · 1/27 · 4.755 = 1.057
Total				12	**H(S) ≈ 3.25 biți**



Verificare: $H(S) \le \bar{l} < H(S) + 1$
$3.25 \le 3.30 < 4.25$ ✓
Huffman e foarte aproape de limita teoretică.

Caracteristicile codării Huffman

1 Perechile frunză

- Cele **două simboluri cel mai puțin frecvente**, unite la același nod, au coduri de **aceeași lungime**, care diferă doar la **ultimul bit**

't'	0	0	0	0	0	'u'	0	0	0	0	1
'm'	0	0	0	1	0	'g'	0	0	0	1	1
'␣'	0	1	0	0	0	'p'	0	1	0	0	1

2 Lungime ↔ frecvență

- Lungimea codului depinde de **probabilitatea relativă** de apariție
- 'a' (6/27) și 'r' (5/27) → **2 biți**
- 't','u','m','g','␣','p' (1/27) → **5 biți**

3 Instantaneu decodabil

- **Niciun** cuvânt de cod nu este prefix pentru altul (simbolurile stau doar în **frunze**)
- → **nicio ambiguitate** la decodare

Observație: în arbore, un simbol nu poate fi pe drumul altui simbol, pentru că simbolurile sunt **numai în frunze** – de aici proprietatea de prefix.

Codificarea textului „programarea calculatoarelor”

Fiecare caracter se înlocuiește cu cuvântul său de cod, iar codurile se **concatenează**:

p	r	o	g	r	a	m	a	r
01001	10	011	00011	10	11	00010	11	10

e	a	␣	c	a	l	c	u	l
0010	11	01000	0101	11	0011	0101	00001	0011

a	t	o	a	r	e	l	o	r
11	00000	011	11	10	0010	0011	011	10

Șirul de biți rezultat (89 biți):

```
01001100110001110110001011100010110100001011
00110101000010011110000001111100010001101110
```

Verificat: concatenarea coincide cu șirul din curs.

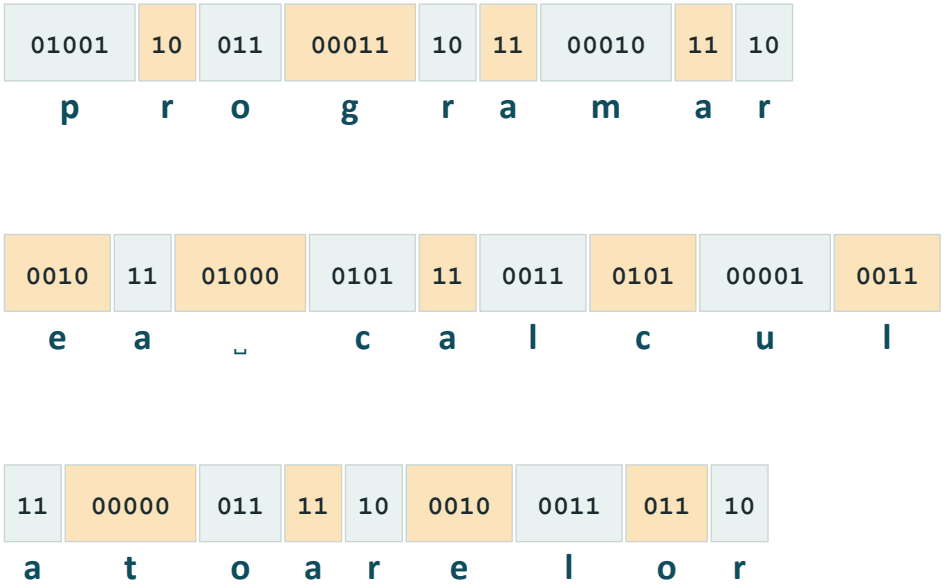
Decodarea Huffman – pas cu pas

Regula: pornim din **rădăcină** și urmăm ramurile după biți (0 = stânga, 1 = dreapta) până la o **frunză** → scriem simbolul și ne întoarcem în rădăcină. Repetăm până se termină biții.

Bit citit	Unde ajungem	Frunză?
0	16/27	nu
1	7/27	nu
0	4/27	nu
0	2/27	nu
1	'p'	**DA → p**
1	11/27 (din rădăcină)	nu
0	'r'	**DA → r**
0	16/27	nu
1	7/27	nu
1	'o'	**DA → o**

Primii 10 biți 0100110011 → „pro”

Segmentarea întregului șir:



Observație: codul fiind **prefix**, nu e nevoie de separatori între cuvintele de cod – granițele se descoperă singure.

Teorema Kraft-McMillan – verificare pe codul Huffman

Un cod cu lungimile $l(s_1), \ldots, l(s_n)$ poate fi unic decodabil dacă: $\Sigma_{k=1..n}\ 2^{-l(sk)} \leq 1$ (5)

Pas 1. Grupăm simbolurile după lungimea codului:

Lungime l	Simboluri	Câte	2^{-l}	Contribuție
2	a, r	2	1/4	2/4 = 16/32
3	o	1	1/8	1/8 = 4/32
4	e, c, l	3	1/16	3/16 = 6/32
5	p, g, m, ␣, u, t	6	1/32	6/32

Pas 2. Adunăm:

$6 \cdot 2^{-5} + 2 \cdot 2^{-2} + 2^{-3} + 3 \cdot 2^{-4}$
$= (6 + 16 + 4 + 6) / 32$
$= 32/32 = \mathbf{1} \leq 1$ ✓

Concluzie: suma = 1 exact: arborele Huffman este **complet** (fiecare nod intern are 2 fii) – nu se irosește niciun cod.

Atenție: inegalitatea este o condiție **necesară**: dacă e respectată, **există** un cod prefix cu acele lungimi; un cod anume o poate respecta și totuși să fie ambiguu (ex.: 1, 01, 010, 000 are suma 1, dar nu e unic decodabil).

5

Codificarea aritmetică

Întreaga secvență de simboluri este transformată într-un singur număr din intervalul [0, 1)

Ideea codării aritmetice și algoritmul

Nu atribuie un cuvânt de cod fiecărui simbol, ci transformă **întreaga secvență** într-o **valoare numerică** din [0,1), a cărei reprezentare ($\log_2$) este mai aproape de **entropia sursei**. Intervalul [0,1) se împarte în subintervale **proporționale cu probabilitățile** (funcție cumulativă).

1 Start

Intervalul curent = **[0, 1)**; considerăm primul simbol.

→

2 Divizare

Împărțim intervalul curent **proporțional cu probabilitățile** simbolurilor.

→

3 Selectare

Alegem subintervalul **simbolului curent** → devine noul interval curent.

→

4 Stop / continuă

Dacă secvența s-a terminat, codul = **orice număr din ultimul interval** (de obicei **mijlocul**); altfel → pasul 2 cu simbolul următor.

← *buclă: pentru fiecare simbol următor*

Formula de divizare: pentru intervalul [low, high): limită = low + (high − low) · (probabilitate cumulată)

Codare aritmetică: setarea problemei

Alfabet $\{s_1, s_2, s_3\}$ cu $p(s_1) = 0.3$, $p(s_2) = 0.5$, $p(s_3) = 0.2$. Secvența de codat: $\mathbf{s_2\ s_3\ s_2\ s_2}$.

Simbol	Probabilitate	Cumulat înainte	Cumulat după	Subinterval în [0,1)
s_1	0.3	0	0.3	[0, 0.3)
s_2	0.5	0.3	0.8	[0.3, 0.8)
s_3	0.2	0.8	1.0	[0.8, 1)

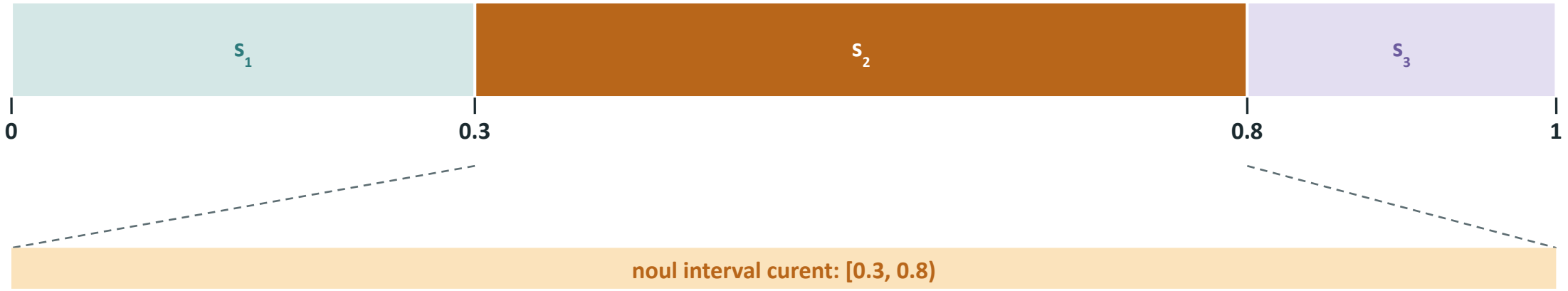
Divizarea inițială a intervalului [0,1):



Lungimea fiecărui subinterval = probabilitatea simbolului. Aceeași împărțire (0.3 | 0.5 | 0.2) se aplică la fiecare pas, în interiorul intervalului curent.

Codare aritmetică – pasul 1: simbolul s_2

Secvența: s_2 s_3 s_2 s_2 [0, 1)



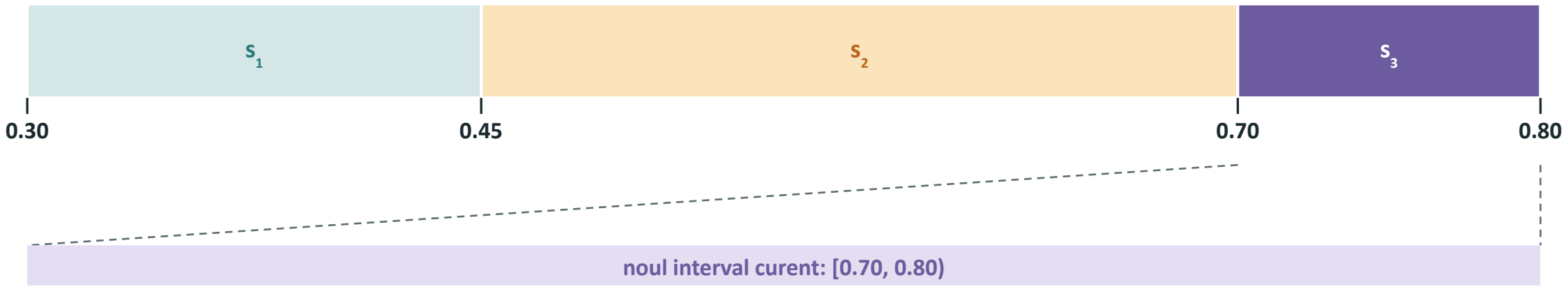
a Interval curent [0, 1), lungime 1

b s_1: [0, 0.3) s_2: [0.3, 0.8) s_3: [0.8, 1)

c Simbolul 1 = **s_2** → nou interval **[0.3, 0.8)**

Codare aritmetică – pasul 2: simbolul s_3

Secvența: s_2 s_3 s_2 s_2 [0.3, 0.8)



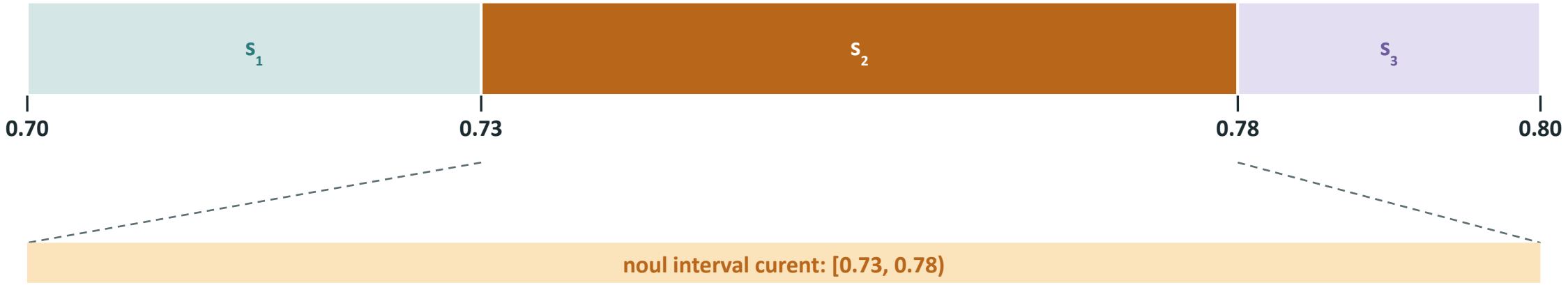
a Interval curent [0.3, 0.8), lungime 0.5

b 0.3 + 0.5·0.3 = **0.45**; 0.3 + 0.5·0.8 = **0.70**

c Simbolul 2 = s_3 → nou interval **[0.70, 0.80)**

Codare aritmetică – pasul 3: simbolul s_2

Secvența: s_2 s_3 s_2 s_2 [0.7, 0.8)



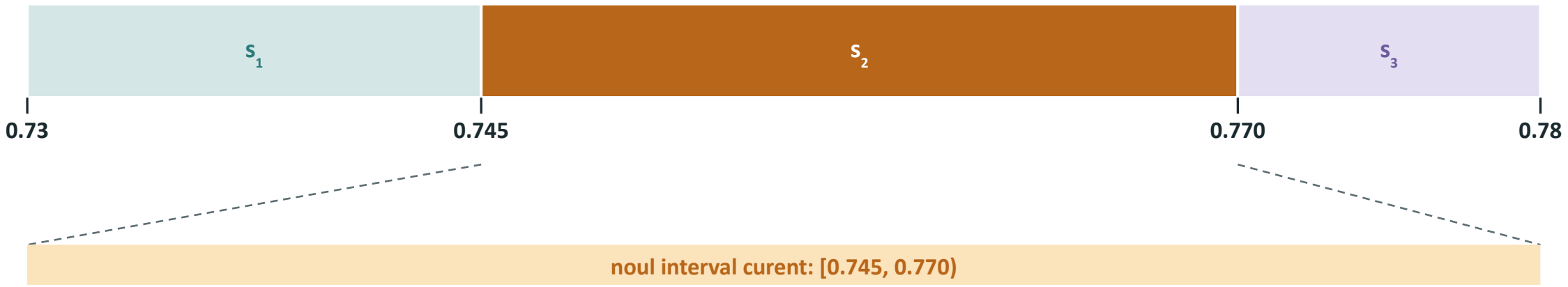
a Interval curent [0.70, 0.80), lungime 0.1

b 0.7 + 0.1·0.3 = **0.73**; 0.7 + 0.1·0.8 = **0.78**

c Simbolul 3 = **s_2** → nou interval **[0.73, 0.78)**

Codare aritmetică – pasul 4: simbolul s_2

Secvența: s_2 s_3 s_2 **s_2** [0.73, 0.78)

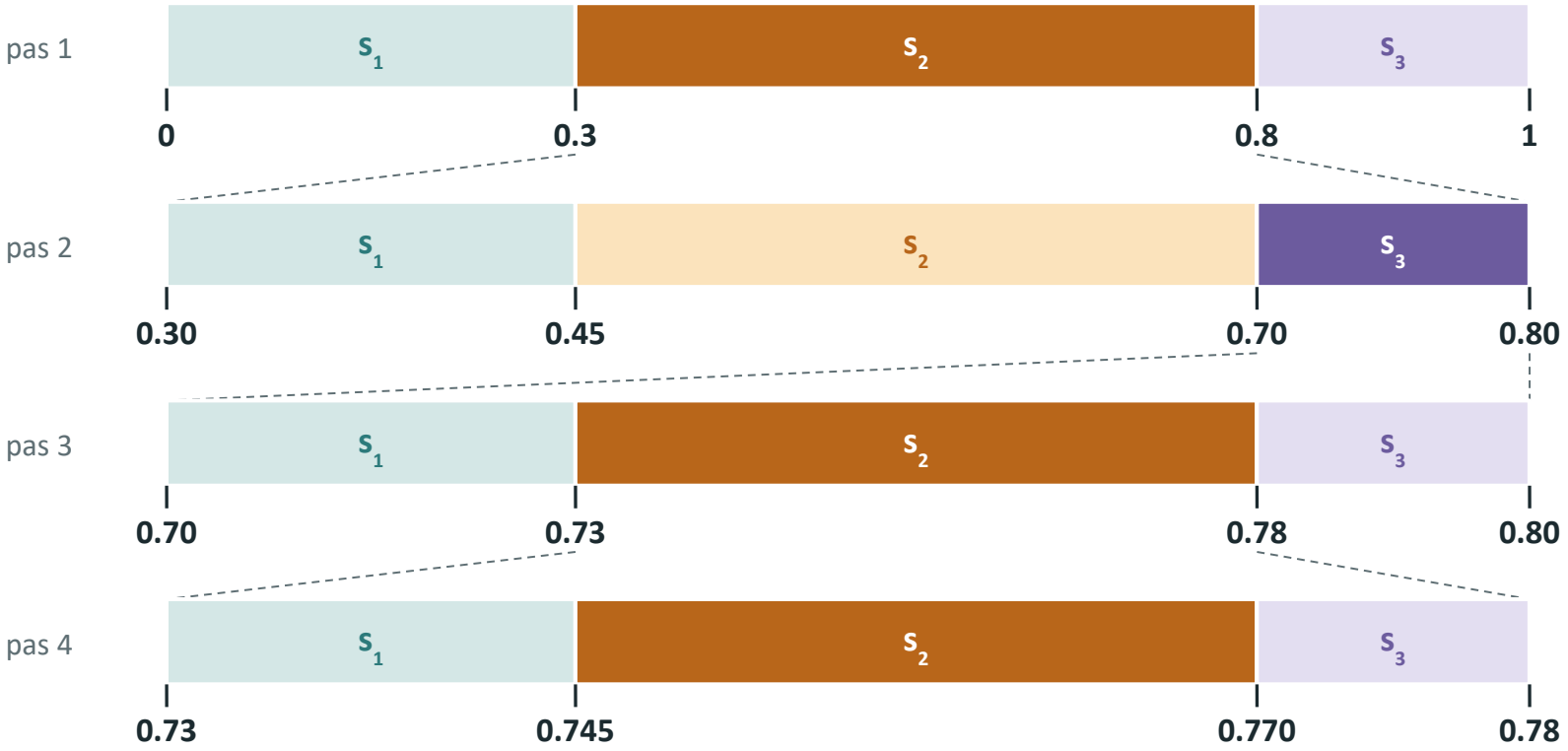


a Interval curent [0.73, 0.78), lungime 0.05

b 0.73 + 0.05·0.3 = **0.745**; 0.73 + 0.05·0.8 = **0.770**

c Simbolul 4 = **s_2** → interval final **[0.745, 0.770)**

Codare aritmetică – imaginea de ansamblu și rezultatul



Rezultat

Interval final: **[0.745, 0.770)**
Orice număr din el codifică secvența.
De obicei se alege **mijlocul**:
(0.745 + 0.770)/2 = **0.7575**

Lungimea intervalului final = 0.5·0.2·0.5·0.5 = **0.025** = produsul probabilităților.

Observație: cu cât secvența e mai probabilă, cu atât intervalul final e mai larg și numărul se scrie cu **mai puțini biți** (≈ $\log_2(1/0.025)$ ≈ 5.3 biți aici, față de 4·2 = 8 biți la lungime fixă) – de aceea codarea aritmetică se apropie de entropie.

Decodarea aritmetică – pas cu pas (numărul 0.75)

Decodorul cunoaște probabilitățile și aplică **aceeași divizare**; la fiecare pas caută subintervalul care **conține** numărul primit.

Pas	Interval curent	Subintervale s_1 | s_2 | s_3	0.75 cade în	Simbol decodat
1	[0, 1)	[0, 0.3) | [0.3, 0.8) | [0.8, 1)	[0.3, 0.8)	s_2
2	[0.3, 0.8)	[0.3, 0.45) | [0.45, 0.70) | [0.70, 0.80)	[0.70, 0.80)	s_3
3	[0.70, 0.80)	[0.70, 0.73) | [0.73, 0.78) | [0.78, 0.80)	[0.73, 0.78)	s_2
4	[0.73, 0.78)	[0.73, 0.745) | [0.745, 0.770) | [0.770, 0.78)	[0.745, 0.770)	s_2

Secvența decodată: $s_2\ s_3\ s_2\ s_2$ ✓

Observație: ca să știe **când să se oprească**, decodorul fie cunoaște **dimensiunea înregistrării** (aici 4 simboluri), fie secvența conține un **simbol marcator de sfârșit**. (Cursul folosește 0.75 la decodare; orice număr din [0.745, 0.770), inclusiv 0.7575, dă același rezultat.)

6

Codificarea Shannon-Fano

Împărțire recursivă „top-down” în submulțimi cu probabilități aproximativ egale

Algoritmul Shannon-Fano

1. **Sortare:** Cele n simboluri se sortează **descrescător** după probabilitatea $p(s_i)$.

2. **Împărțire:** Mulțimea se împarte în **două submulțimi** cu sume ale probabilităților **aproximativ egale**.

3. **Atribuire bit:** Simbolurile dintr-o submulțime primesc bitul **1**, cele din cealaltă bitul **0** (în curs: grupa de sus = 1).

4. **Recursivitate:** Fiecare submulțime se împarte din nou; următorul bit se atribuie la fel.

5. **Oprire:** Procedeul continuă până când fiecare submulțime conține **un singur simbol**.

Huffman vs. Shannon-Fano

- **Huffman**: de jos în sus (bottom-up), unește cele mai mici probabilități
- **Shannon-Fano**: de sus în jos (top-down), împarte mulțimea în două
- Ambele produc **coduri prefix** (instantaneu decodabile)
- Huffman garantează lungimea medie **minimă**; Shannon-Fano e adesea foarte aproape

Shannon-Fano – etapa 1 din 4

Simbol	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8	s_9	s_{10}
p	0.25	0.15	0.15	0.07	0.07	0.07	0.06	0.06	0.06	0.06
Etapa 1	**1** (Σ=0.55)			**0** (Σ=0.45)						
Cod parțial	1	1	1	0	0	0	0	0	0	0

1 Total 1.00. Tăiem după s_3: $\{s_1,s_2,s_3\} = 0.55$ vs. $\{s_4 \ldots s_{10}\} = 0.45$ (diferență 0.10). Alternative: după s_2 → 0.40/0.60 (dif. 0.20); după s_4 → 0.62/0.38 (dif. 0.24). Prima grupă primește **1**, a doua **0**.

Shannon-Fano – etapa 2 din 4

Simbol	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8	s_9	s_{10}
p	0.25	0.15	0.15	0.07	0.07	0.07	0.06	0.06	0.06	0.06
Etapa 1	**1** (Σ=0.55)			**0** (Σ=0.45)						
Etapa 2	**1** (Σ=0.25)	**0** (Σ=0.30)		**1** (Σ=0.21)			**0** (Σ=0.24)			
Cod parțial	11	10	10	01	01	01	00	00	00	00

2

$\{s_1,s_2,s_3\}$: s_1 = 0.25 vs. $\{s_2,s_3\}$ = 0.30 (dif. 0.05) → **11** / **10**. $\{s_4…s_{10}\}$: $\{s_4,s_5,s_6\}$ = 0.21 vs. $\{s_7…s_{10}\}$ = 0.24 (dif. 0.03) → **01** / **00**.

Shannon-Fano – etapa 3 din 4

Simbol p	s_1 0.25	s_2 0.15	s_3 0.15	s_4 0.07	s_5 0.07	s_6 0.07	s_7 0.06	s_8 0.06	s_9 0.06	s_{10} 0.06
Etapa 1	**1** (Σ=0.55)						**0** (Σ=0.45)			
Etapa 2	**1** (Σ=0.25)	**0** (Σ=0.30)		**1** (Σ=0.21)			**0** (Σ=0.24)			
Etapa 3		**1** (Σ=0.15)	**0** (Σ=0.15)	**1** (Σ=0.07)	**0** (Σ=0.14)		**1** (Σ=0.12)		**0** (Σ=0.12)	
Cod parțial	11	101	100	011	010	010	001	001	000	000

3

s_2 | s_3 → **101** / **100**. $\{s_4,s_5,s_6\}$: s_4 = 0.07 vs. $\{s_5,s_6\}$ = 0.14 → **011** / **010**. $\{s_7 \ldots s_{10}\}$: 0.12 vs. 0.12 → **001** / **000**. s_1 (11) e deja singur – nu se mai împarte.

Shannon-Fano – etapa 4 din 4

Simbol / p	s_1 0.25	s_2 0.15	s_3 0.15	s_4 0.07	s_5 0.07	s_6 0.07	s_7 0.06	s_8 0.06	s_9 0.06	s_{10} 0.06
Etapa 1	**1** (Σ=0.55)			**0** (Σ=0.45)						
Etapa 2	**1** (Σ=0.25)	**0** (Σ=0.30)		**1** (Σ=0.21)			**0** (Σ=0.24)			
Etapa 3		**1** (Σ=0.15)	**0** (Σ=0.15)	**1** (Σ=0.07)	**0** (Σ=0.14)		**1** (Σ=0.12)		**0** (Σ=0.12)	
Etapa 4					**1** (Σ=0.07)	**0** (Σ=0.07)	**1** (Σ=0.06)	**0** (Σ=0.06)	**1** (Σ=0.06)	**0** (Σ=0.06)
Cod parțial	11	101	100	011	0101	0100	0011	0010	0001	0000

4 Ultimele perechi se separă: s_5/s_6 → **0101** / **0100**; s_7/s_8 → **0011** / **0010**; s_9/s_{10} → **0001** / **0000**. Fiecare submulțime are acum un singur simbol → STOP.

Shannon-Fano – rezultat, lungime medie și entropie

Simbol	p	Cod	l	p · l	$\log_2(1/p)$	$p \cdot \log_2(1/p)$
s_1	0.25	11	2	0.50	2.000	0.500
s_2	0.15	101	3	0.45	2.737	0.411
s_3	0.15	100	3	0.45	2.737	0.411
s_4	0.07	011	3	0.21	3.837	0.269
s_5	0.07	0101	4	0.28	3.837	0.269
s_6	0.07	0100	4	0.28	3.837	0.269
s_7	0.06	0011	4	0.24	4.059	0.244
s_8	0.06	0010	4	0.24	4.059	0.244
s_9	0.06	0001	4	0.24	4.059	0.244
s_{10}	0.06	0000	4	0.24	4.059	0.244
Σ	1.00			**3.13**		**3.10**

Lungimea medie

$0.25 \cdot 2 + 0.15 \cdot 3 + 0.15 \cdot 3 + 0.07 \cdot 3 + 0.07 \cdot 4 + 0.07 \cdot 4 + 4 \cdot (0.06 \cdot 4) =$ **3.13 biți**

Entropia

$0.5 + 0.30 \cdot 2.737 + 0.21 \cdot 3.837 + 0.24 \cdot 4.059 = 0.5 + 0.821 + 0.806 + 0.974 \approx$ **3.10 biți**

Concluzie: 3.13 e foarte aproape de 3.10 → cod **performant**. (Lungime fixă: 4 biți. Huffman dă tot 3.13 pentru această sursă.)

7

Codificarea informației multimedia; transmiterea datelor în funcție de tip

Constrângeri, tipuri de redundanță, reprezentarea imaginilor, codare entropică vs. codare cu informații despre sursă

De ce trebuie comprimată informația multimedia?

Informația multimedia necesită

- Soluții pentru **depozitarea** unor volume mari de date
- Medii de transmisie cu **lățime de bandă** mare (nr. maxim de biți transferați pe secundă)

Soluția

Compresia (prin codificare) **reduce numărul de biți transferați** fără a afecta conținutul informației.

Constrângerile codificării multimedia

1 Calitate

Calitatea (ex. a unei imagini) să nu aibă de suferit în urma compresiei/decompresiei.

2 Timp și complexitate

Timpul de execuție și complexitatea tehnicilor de codare/decodare trebuie să fie minime.

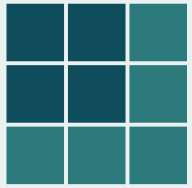
3 Formatul

Unele informații se codifică în format **continuu** (funcție de timp), altele în format **discret**.

Tipuri de redundanță în sunet, imagine și video

1 Redundanță spațială

Un pixel are aceeași culoare sau culori foarte apropiate de **pixelii învecinați** → corelație între vecini.



2 Redundanță temporală

În video, conținutul se schimbă puțin **de la un cadru la altul** → corelație între cadre succesive.

3 Redundanță spectrală

Imaginea color = **3 plane** (R, G, B); apare corelație între **planele de culoare**. (Imaginea gri = 1 plan.)

4 Redundanță vizuală

Datorată proprietăților de **percepție ale sistemului vizual uman** (ochiul nu observă unele detalii).



Reprezentarea imaginilor: tonuri de gri și RGB

Imaginea = **tablou bidimensional de pixeli**, fiecare cu o culoare. Cu cât sunt mai mulți pixeli, cu atât **claritatea** crește.

Tonuri de gri

Cu **n biți** de precizie per pixel (uzual 8, 16 sau 32) → **2^n** tonuri de gri.
Ex.: 8 biți → 2^8 = **256** tonuri.



0 (negru) 255 (alb)

pixel = 8 biți:

1	0	1	1	0	1	0	0

= 180 (gri deschis)

Color – RGB

Fiecare canal (**R**oșu, **V**erde, **A**lbastru) e o matrice separată cu intensitățile culorii.
8 biți/canal → $2^8 \cdot 2^8 \cdot 2^8 = 2^{24} \approx$ **16.77 milioane** de culori (mai mult decât percepe ochiul).

R	1	1	1	1	1	1	1	1
G	0	0	0	0	0	0	0	0
B	0	0	0	0	0	0	0	0

= roșu pur

00000000 = intensitate zero (culoare absentă); 11111111 = intensitate maximă

Două categorii de tehnici de compresie multimedia

Tehnici de compresie (codare) multimedia

Codare entropică → FĂRĂ pierderi

Nu ține cont de natura informației: datele sunt simple secvențe de biți (caractere, pixeli, sunete…) transformate în secvențe **mai scurte**.

1. RLE (Run-Length Encoding)
2. Eliminarea repetiției
3. Înlocuirea modelelor
4. Codarea Huffman

Codare sursă → CU pierderi

Ține cont de **tipul semnalului** (audio, video) și de **conținut**; folosește proprietățile lui pentru un **raport de compresie mai bun** (codare semantică).

1. Compresia cu transformate (JPEG, MPEG)
2. Cuantizarea vectorială
3. Codarea diferențială / predictivă
4. Eșantionarea

Codare entropică (1): RLE pentru imagini

RLE mapează o secvență de pixeli $x_1, x_2, \ldots, x_n$ într-o secvență de perechi $\mathbf{(c_1, l_1), \ldots, (c_k, l_k)}$, unde c_i = culoarea (intensitatea), l_i = lungimea secvenței i de pixeli cu aceeași intensitate.

Pas 1. Secvența de 73 pixeli (1 = Roșu, 2 = Verde, 3 = Albastru):



Pas 2. Numărăm fiecare secvență:

10 × 2 (Verde) **15 × 1 (Roșu)** **20 × 3 (Albastru)** **28 × 1 (Roșu)**

Pas 3. Rezultatul codării:

`(2, 10)` `(1, 15)` `(3, 20)` `(1, 28)` **73 de valori → 4 perechi**

Obs.: în compresie, „imagine” = ilustrație statică; „video” = secvență de imagini (imagini în mișcare).

Codare entropică (2): repetiții și modele

Eliminarea repetiției („Repetition Suppression”)

n apariții succesive ale unui caracter → un **caracter semnal (flag)** urmat de **numărul de apariții**. Utilă la eliminarea zerourilor din fișiere de date / bitmap sau a spațiilor din text și cod sursă.



4 cifre + **30 de zerouri** (34 caractere)



5689 + flag „f” + 30 → **5689f30** (7 caractere)

Înlocuirea modelelor („Pattern Substitution”)

Fiecare **model des repetat** (cuvinte, secvențe de cuvinte) e înlocuit printr-un **cod**; modelele mai frecvente primesc coduri **mai scurte**.

Huffman în multimedia

Pe baza frecvenței **simbolurilor** (text) sau a **octeților** (imagini). Lista de coduri: per imagine sau per mulțime de imagini (video). Folosit în **JPEG**, **MPEG-1 … MPEG-4**.

Codare sursă (1): compresia cu transformate

Informația suferă o **transformare matematică** (ex. **transformata Fourier** sau **transformata cosinus**) → reorganizare după **frecvențe**: din domeniul timp/spațiu în domeniul frecvențelor.

f(x, y)
imaginea
x, y = variabile spațiale

transformata →

F(u, v)
spectrul Fourier
u, v = frecvențe spațiale

Reprezentarea spectrală surprinde **modificările rapide** de culoare sau luminozitate.

JPEG (Joint Photographic Experts Group) – lanțul de compresie:

Transformata cosinus → **Cuantizare** → **Codare RLE** → **Codare Huffman**

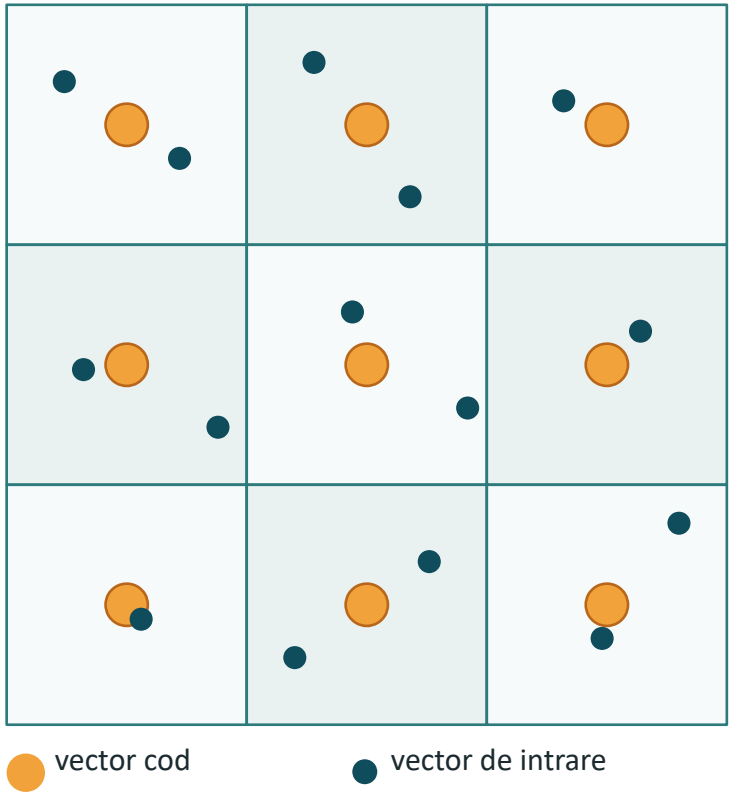
Imagini gri sau color cu **tonuri continue** (milioane de culori); raport ales de utilizator, **~1/10 – 1/20**. Ex.: fax color, imagistică medicală, DTP, arte grafice.

MPEG

Compresie **video**: imagini + sunet + **sincronizarea** lor. Există mai multe standarde.

Codare sursă (2): cuantizarea vectorială (VQ)

Mapare a unui vector de intrare **x** = (x_1, …, x_n) din X^n într-un vector **y** din mulțimea discretă de aproximări **Y = {y_1, …, y_m}** (= **dicționar**, vectori cod). n = dimensiunea cuantizatorului.



Schemă 2D: fiecare zonă S_i are un singur vector cod

Partiția spațiului în M zone S_i

$\cup S_i = X^n$ (zonele acoperă tot spațiul)

$S_i \cap S_j = \emptyset$, $i \neq j$ (zonele nu se suprapun)

$y_i \in S_i$ (fiecare zonă conține un singur cuvânt de cod)

În imagini

Imaginea → **blocuri de pixeli** = vectori de intrare. Fiecare bloc se înlocuiește cu **indicele** vectorului din dicționar cel mai apropiat (distorsiune minimă), după **distanța Euclidiană**:

$d(x, y_i) = \sqrt{(\Sigma_{j=1..n} (x_j - y_{ij})^2)}$

Cuantizare vectorială – exemplu numeric pas cu pas

Exemplu ilustrativ (nu este în curs): dicționar cu 3 vectori cod în 2D și vectorul de intrare **x = (3, 4)**.

Vector cod	$x - y_i$	$(\Delta_1)^2 + (\Delta_2)^2$	$d(x, y_i)$
$y_1 = (0, 0)$	(3, 4)	9 + 16 = 25	5.00
$y_2 = (4, 4)$	(−1, 0)	1 + 0 = 1	**1.00 ← minim**
$y_3 = (8, 0)$	(−5, 4)	25 + 16 = 41	6.40

1. Calculăm diferențele pe fiecare coordonată
2. Ridicăm la pătrat și adunăm
3. Extragem radicalul → distanța
4. Alegem vectorul cu distanța minimă: y_2
5. Transmitem doar **indicele 2** în locul lui (3, 4)

Observație: receptorul reconstruiește (4, 4) în loc de (3, 4) – o mică **pierdere** de informație, în schimbul transmiterii unui simplu indice → compresie **cu pierderi**.

Codare sursă (3): codare diferențială și eșantionare

Δ Codarea diferențială / predictivă

Valorile pixelilor **învecinați** sunt corelate → valoarea unui pixel poate fi **aproximată** dintr-o combinație a valorilor unor vecini ai săi (codificați anterior).

∿ Eșantionarea

Se transmit **numai anumite secvențe de pixeli** care conțin suficientă informație. La sunet: digitizarea **eșantionează** unda continuă (nr. eșantioane/secundă); eșantioanele adiacente sunt corelate.

Metoda prefix + sufix: fiecare pixel p se compară cu un **pixel de referință** (un vecin imediat, deja codificat):

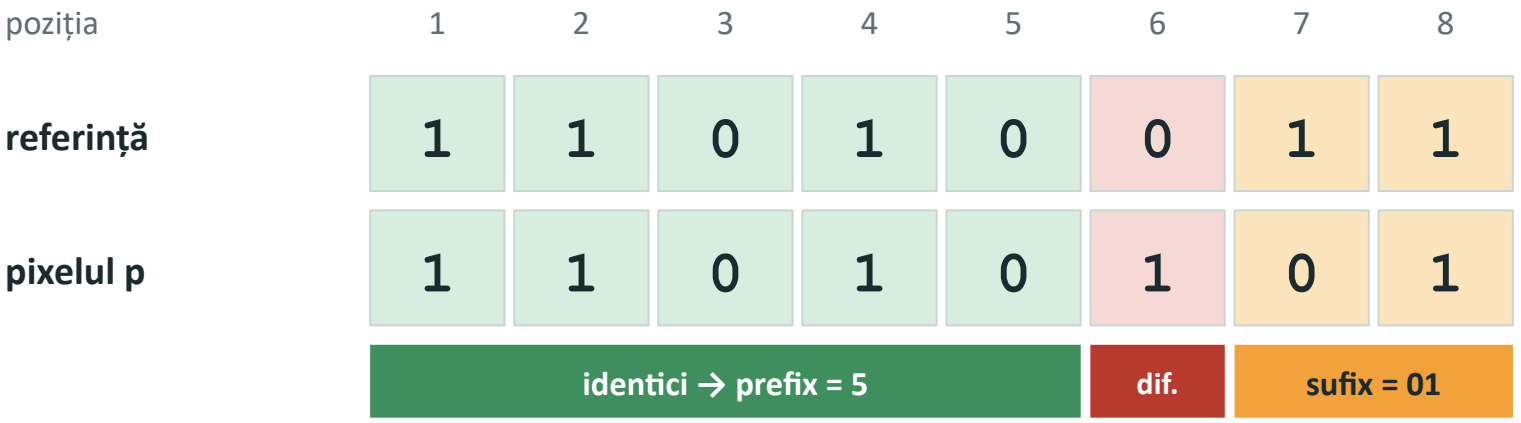
Prefix

numărul de biți cei mai semnificativi ai lui p **identici** cu cei ai pixelului de referință

Sufix

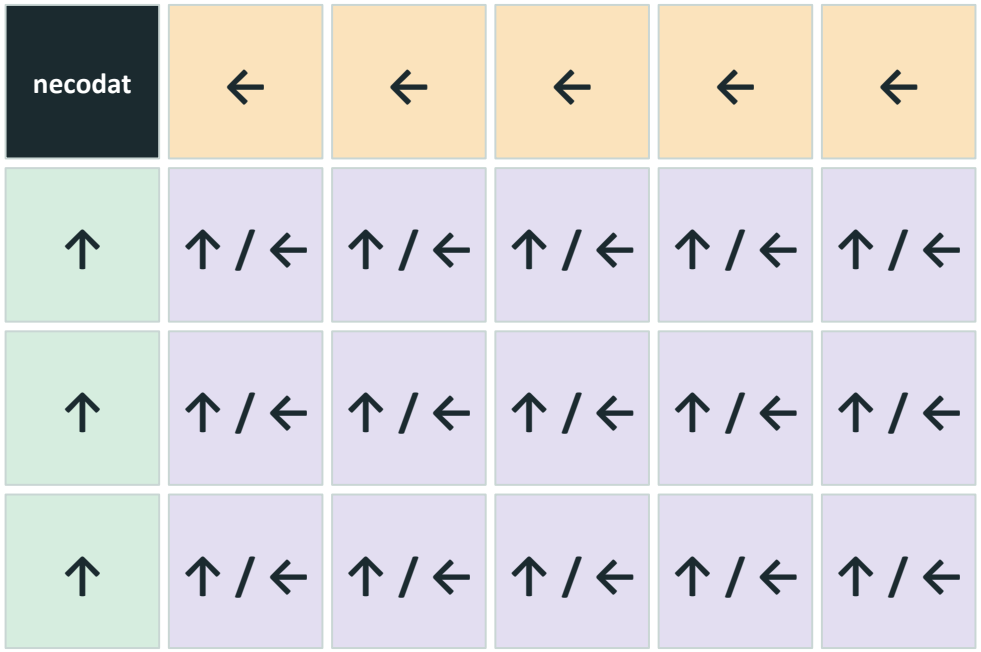
restul biților cei mai puțin semnificativi ai lui p (după poziția primei diferențe)

Exemplu prefix + sufix – pas cu pas



1. Comparăm bit cu bit de la stânga (cel mai semnificativ): pozițiile 1–5 coincid (11010).
2. La poziția 6 apare prima diferență: referința are 0, p are 1.
3. Prefix = **5** (numărul de biți identici).
4. Sufix = **01** (pozițiile 7 și 8). Bitul 6 **nu se transmite**: decodorul știe că acolo e prima diferență, deci bitul lui p este **inversul** bitului referinței (0 → 1).
5. Cod(p) = (5, 01). Decodare: 11010 (din referință) + 1 (inversat) + 01 = **11010101** ✓

Alegerea pixelului de referință



Imagine (pixeli parcurși linie cu linie)

Primul pixel al imaginii
rămâne **necodat**

Ceilalți pixeli din prima linie
referința = vecinul **din stânga** (←)

Primul pixel din liniile următoare
referința = pixelul **de deasupra** (↑)

Toți ceilalți pixeli
referința = pixelul **de deasupra** sau cel **din stânga**

Concluzie: valoarea pixelului curent este **prezisă** pe baza pixelilor vecini **codificați anterior** – de aceea decodorul o poate reface.

?

Test de autoevaluare – rezolvări pas cu pas

1. Arborele Huffman pentru S = $\{s_1 \ldots s_8\}$ (variante) 2. Câte arborescențe Huffman există? 3. De ce depinde lungimea cuvântului de cod?

Problema 1 – enunț și datele

Alfabet S = {s_1, …, s_8} cu probabilitățile de mai jos. Construiți arborele Huffman. Există mai multe variante?

Simbol	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8
p (/30)	1	1	1	2	3	4	4	14

Lista ordonată crescător: s_1:1, s_2:1, s_3:1, s_4:2, s_5:3, s_6:4, s_7:4, s_8:14

Observație: când mai multe noduri au **aceeași probabilitate** (ex. trei de 1/30, sau 2/30 cu s_4), putem alege oricare dintre ele → rezultă **arbori diferiți**. Vom construi două variante și vom compara lungimile medii.

Varianta 1

Unim s_1+s_2, apoi s_3+s_4, …

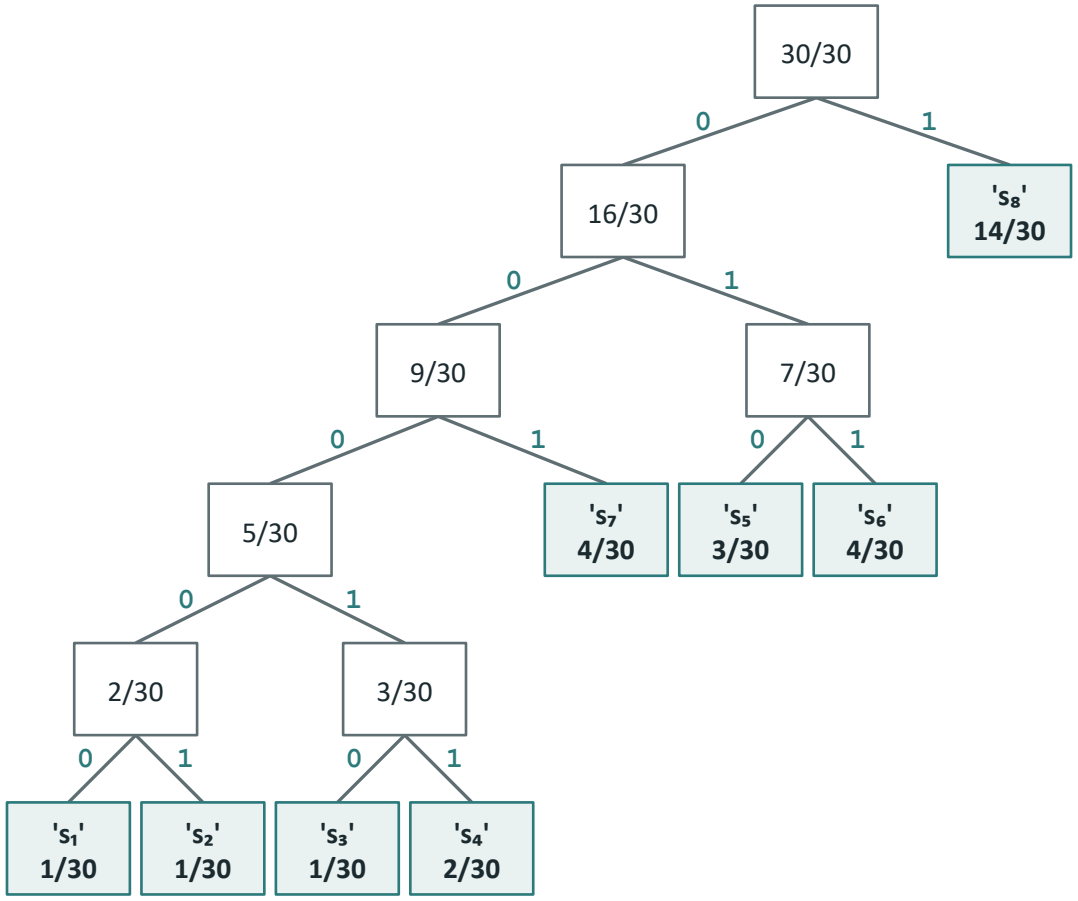
Varianta 2

Unim s_2+s_3, apoi s_1+(nodul 2/30), …

Problema 1 – Varianta 1 (pas cu pas)

Unire	Lista înainte (ordonată)	Se unesc	Nod nou
1	s1 s2 s3 s4 s5 s6 s7 s8 (1,1,1,2,3,4,4,14)	s1 + s2	2/30
2	s3:1 s4:2 [2] s5:3 s6:4 s7:4 s8:14	s3 + s4	3/30
3	[2] [3] s5:3 s6:4 s7:4 s8:14	[2] + [3]	5/30
4	s5:3 s6:4 s7:4 [5] s8:14	s5 + s6	7/30
5	s7:4 [5] [7] s8:14	[5] + s7	9/30
6	[7] [9] s8:14	[9] + [7]	16/30
7	s8:14 [16]	[16] + s8	30/30

Simbol	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8
Cod	00000	00001	00010	00011	010	011	001	1

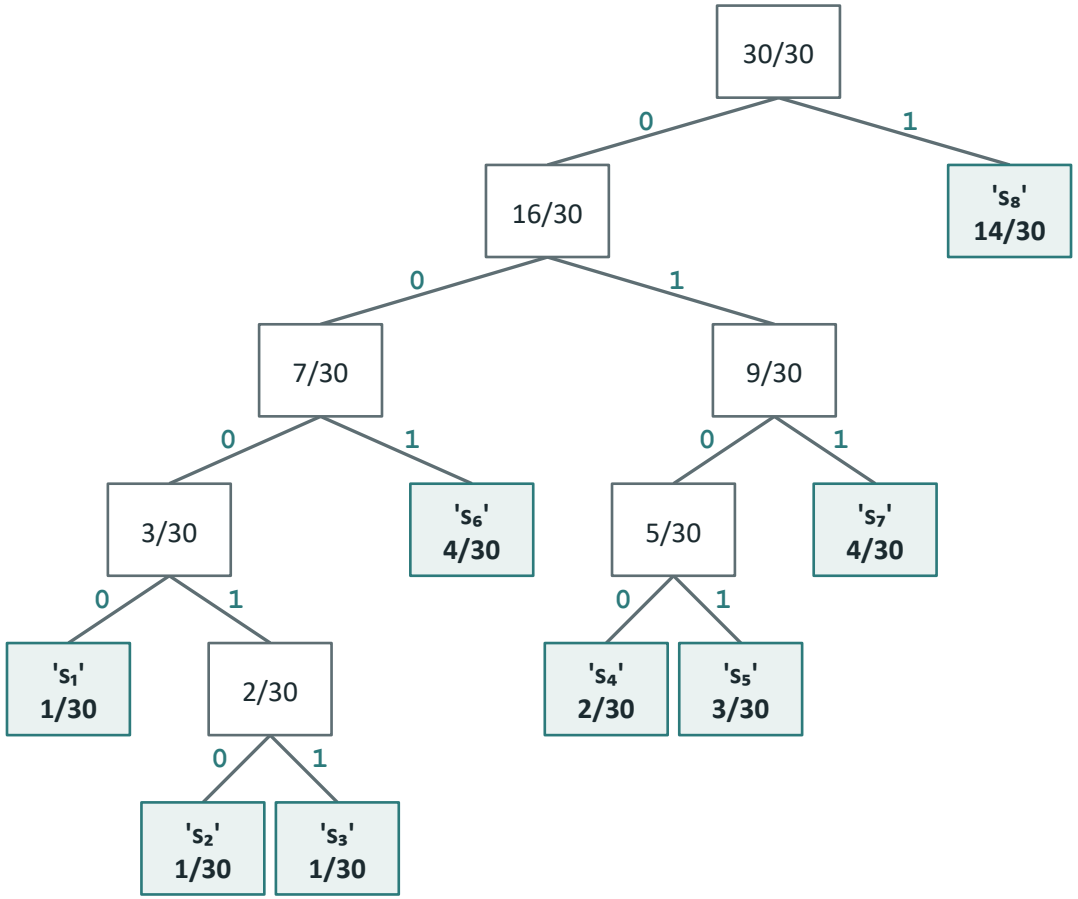


$\bar{l} = (1\cdot5+1\cdot5+1\cdot5+2\cdot5+3\cdot3+4\cdot3+4\cdot3+14\cdot1)/30 =$ **72/30 = 2.4 biți**

Problema 1 – Varianta 2 (pas cu pas)

Unire	Lista înainte (ordonată)	Se unesc	Nod nou
1	s1 s2 s3 s4 s5 s6 s7 s8 (1,1,1,2,3,4,4,14)	s2 + s3	2/30
2	s1:1 s4:2 [2] s5:3 s6:4 s7:4 s8:14	s1 + [2]	3/30
3	s4:2 [3] s5:3 s6:4 s7:4 s8:14	s4 + s5	5/30
4	[3] s6:4 s7:4 [5] s8:14	[3] + s6	7/30
5	s7:4 [5] [7] s8:14	[5] + s7	9/30
6	[7] [9] s8:14	[7] + [9]	16/30
7	s8:14 [16]	[16] + s8	30/30

Simbol	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8
Cod	0000	00010	00011	0100	0101	001	011	1



$\bar{l} = (1\cdot4+1\cdot5+1\cdot5+2\cdot4+3\cdot4+4\cdot3+4\cdot3+14\cdot1)/30 =$ **72/30 = 2.4 biți**

Problema 1 – concluzii

	Varianta 1	Varianta 2
Lungimi coduri $s_1 \ldots s_8$	5, 5, 5, 5, 3, 3, 3, 1	4, 5, 5, 4, 4, 3, 3, 1
Lungime medie $\bar{l}$	**2.4 biți**	**2.4 biți**
Kraft: $\Sigma\, 2^{-l}$	4/32 + 3/8 + 1/2 = 1	2/16+2/32+2/16+2/8+1/2 = 1

Da, există mai multe variante

- Alegeri diferite la **egalitate de probabilități** → arbori cu **forme diferite**
- Codurile individuale diferă (ex. s_1: 00000 vs. 0000)
- Dar **lungimea medie e aceeași** – ambele sunt optime

Verificare cu entropia

$H(S) = \Sigma\, p \cdot \log_2(1/p) \approx$ **2.37 biți**

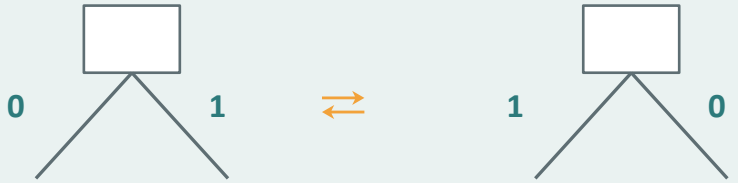
$2.37 \leq 2.4 < 3.37$ ✓

Lungime fixă (8 simboluri): 3 biți.

Problemele 2 și 3

2 2. Câte arborescențe Huffman?

- Pentru n simboluri, arborele Huffman are **n – 1 noduri interioare**
- Fiecare nod interior are 2 ramuri etichetate 0 / 1, dar etichetarea **nu este fixă** – se poate inversa
- 2 alegeri pentru fiecare din cele n – 1 noduri → $\mathbf{2^{n-1}}$ posibilități
- Ex.: n = 8 → 2^7 = **128** arborescențe (pentru aceeași structură)



3 3. De ce depinde lungimea cuvântului de cod?

- Depinde de **probabilitatea simbolului**: probabilitate mare → cod scurt
- Dar depinde și de **dimensiunea alfabetului**
- Alfabet mare cu simboluri **egal probabile** → probabilități mici → coduri **lungi** pentru toate
- Dacă unele simboluri au probabilități mari → ele primesc coduri scurte, iar celelalte (implicit mai puțin probabile) → coduri mai lungi

Rezumatul unității

1	**Compresia**	transformă fluxul de intrare într-un flux mai mic; reduce **redundanța**: frecvent → cod scurt, rar → cod lung
2	**Proprietăți**	**nesingular** (coduri distincte) ⊃ **unic decodabil** ⊃ **instantaneu decodabil** (proprietatea de prefix)
3	**Entropia**	$H(S) = \Sigma\ p \cdot \log_2(1/p)$ – limita inferioară: $\mathbf{H \le \bar{l} < H + 1}$
4	**Fără pierderi**	**RLE**, **Huffman** (bottom-up), **aritmetică** (un număr în [0,1)), **Shannon-Fano** (top-down)
5	**Multimedia**	redundanță spațială, temporală, spectrală, vizuală; codare **entropică** (lossless) vs. **sursă** (lossy): transformate, VQ, diferențială, eșantionare

Bibliografie: Dinu, S., Pomazan, C. – Programarea calculatoarelor (Ed. Nautica, 2013); Kernighan, B. W., Ritchie, D. M. – Limbajul C (Teora, 2003); Năstac, I. (Printech, 2006); Negrescu, L. (Albastră, 2000); Dinu, S., Vulcu, V. – culegere de probleme (Nautica, 2019).